

Verification Continuum™

FSDB Reader

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	8
Synopsys Statement on Inclusivity and Diversity	8

1. FSDB Reader	10
Overview	11
Opening the FSDB File	12
Traversing Design Hierarchies	12
Multiple Tree Issues	13
Traversing Value Changes	15
Traversing User-Defined Data Types	15
Selecting Variables By Their Id Codes	16
Loading Value Changes for Variables Into Memory	16
Creating Value-Change Traverse Handles for Variables	16
Traversing Value Changes Via Traverse Handles	17
Closing the FSDB File	17
Scope Tree	18
Jump Behavior	19
Recording Dumpoff and Dump on	20
Getting Support Information in the FSDB File	20
Traversing Value Changes in Time-Based Fashion	21
Transaction Traversal	22
Memory Traversal	24
Code Examples	24
read_verilog.cpp	24
tb.cpp	39

2. Function List	49
ffrObject	49
ffrAddToSignalList	49

Contents

ffrCheckFile	50
ffrCheckFSDB	50
ffrClose	51
ffrCreateTimeBasedVCTrvsHdl	51
ffrCreateVCTrvsHdl	52
ffrExtractScaleUnit	52
ffrFreeNavDB	53
ffrGetAllDumperEnvVarVal;	53
ffrGetAnnotation	54
ffrGetAPIVersion	55
ffrGetDumperEnvVarVal;	55
ffrGetDumpOffRange	55
ffrGetEvcdPortStrength	56
ffrGetFileDumperType	57
ffrGetFileSimType	58
ffrGetFileType	58
ffrGetFSDBInfo	58
ffrGetIdent	59
ffrGetInterface	59
ffrGetLoopMarkerArray	60
ffrGetMaxFsdBTag64	61
ffrGetMaxFsdBTagDouble	61
ffrGetMaxFsdBTagFloat	62
ffrGetMaxVarIdcode	62
ffrGetMinFsdBTag64	63
ffrGetMinFsdBTagDouble	63
ffrGetMinFsdBTagFloat	64
ffrGetOEMActiveObj	64
ffrGetOEMFuncTbl	64
ffrGetPropVarStatInfo	65
ffrGetRelationName	65
ffrGetScaleUnit	66
ffrGetScopeSeparator	66
ffrGetSessionListInfo	67
ffrGetSimDate	67
ffrGetSimVersion	68
ffrGetSupportInfo	68
ffrGetTransInfo	70
ffrGetTreeCBFunc	70

Contents

ffrGetTriggerPointVarIdcode	71
ffrGetVarInfoByVarIdcode	72
ffrGetVCCount	73
ffrGetVersion	73
ffrGetVrtlFileType	74
ffrGetVwlsSet	74
ffrGetXTagType	74
ffrHasBeenComputed	75
ffrHasDataTypeDef	76
ffrHasDumpOffRange	76
ffrHasGlitch	76
ffrHasLoopMarkerVar	77
ffrHasNewTree	78
ffrHasScopeTree	78
ffrHasSupportInfo	78
ffrHasTriggerPointVar	79
ffrInfoSuppress	79
ffrInitialize	80
ffrInsertComputedPropVC	80
ffrInsertComputedTransVC	81
ffrInsertComputedVC	82
ffrIsComputedVar	83
ffrIsEvcd	84
ffrIsFSDB	84
ffrIsInSignalList	85
ffrIsMiddleOrComputedVar	85
ffrIsMiddleVar	86
ffrIsThereSeqNum	87
ffrIsViewWindowSet	87
ffrIsVrtlFile	88
ffrLoadSignals	88
ffrNoChangeEpicVarName	89
ffrOpen	89
ffrOpen2	90
ffrOpen3	92
ffrOpenNonSharedObj	92
ffrOver2Gldcode	93
ffrReadDataTypeDefByBlkIdx	93
ffrReadDataTypeDefByBlkIdx2	94

Contents

ffrReadScopeTree	95
ffrReadScopeTree2	96
ffrReadScopeVarTree	96
ffrReadScopeVarTree2	97
ffrReadVarByLogUOff	98
ffrReadVarByLogUOff	99
ffrRegisterErrorCBFunc	99
ffrRegisterInfoCBFunc	100
ffrRegisterWarnCBFunc	101
ffrResetSignalList	102
ffrResetViewWindow	102
ffrSetExitOnMallocFail	103
ffrSetFsdbTag64LoadRange	103
ffrSetLoadNavDBOnlyOnce	104
ffrSetOEMActiveObj	104
ffrSetTreeCBFunc	105
ffrSetViewWindow	105
ffrSync	106
ffrUnloadSignals	107
ffrUnloadSignals	108
ffrUnloadSignals	109
ffrWarnSuppress	109
ffrTimeBasedVCIterOne	110
ffrFree	110
ffrGetBitSize	110
ffrGetByteCount	111
ffrGetBytesPerBit	111
ffrGetGlitchNum	112
ffrGetSeqNum	112
ffrGetVarIdcode	113
ffrGetVarIdcodeXTagVCSeqNum	113
ffrGetVarType	114
ffrGetVC	115
ffrGetXTag	115
ffrGotoNextVC	116
ffrIsItPropVar	117
ffrIsItStreamVar	117
ffrVCIterOne	117

Contents

ffrFree	117
ffrGetBitSize	118
ffrGetByteCount	118
ffrGetBytesPerBit	119
ffrGetGlitchNum	119
ffrGetMaxXTag	120
ffrGetMinXTag	121
ffrGetRefCount	121
ffrGetSeqNum	122
ffrGetTransInfo	123
ffrGetTransVal	123
ffrGetVarType	124
ffrGetVC	124
ffrGetXTag	125
ffrGetXTagVCSeqNum	126
ffrGotoNextVC	127
ffrGotoPrevVC	127
ffrGotoTheFirstVC	127
ffrGotoXTag	128
ffrHasIncoreVC	128
Other	129
void ffrOEMInit	129
void ffrOEMInit2	130

3. Mapping Between Type and Data 132

4. FAQs 135

Preface

The FSDB Reader provides detailed information about the FSDB Reader, its function list, and the mapping between tree callback type and tree callback data. It also provides information regarding some of frequently-asked questions.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to verdi_support@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to verdi_support@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary

Contents

language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

FSDB Reader

This chapter describes the following topics:

- [Overview](#)
- [Opening the FSDB File](#)
- [Traversing Design Hierarchies](#)
- [Multiple Tree Issues](#)
- [Traversing Value Changes](#)
- [Traversing User-Defined Data Types](#)
- [Selecting Variables By Their Id Codes](#)
- [Loading Value Changes for Variables Into Memory](#)
- [Creating Value-Change Traverse Handles for Variables](#)
- [Traversing Value Changes Via Traverse Handles](#)
- [Closing the FSDB File](#)
- [Scope Tree](#)
- [Jump Behavior](#)
- [Recording Dumpoff and Dumpon](#)
- [Getting Support Information in the FSDB File](#)
- [Traversing Value Changes in Time-Based Fashion](#)
- [Transaction Traversal](#)
- [Memory Traversal](#)
- [Code Examples](#)

Overview

An FSDB file may contain the following three types of information:

- Design hierarchies
- User-defined data types
- Value changes

The FSDB Reader notifies an application about design hierarchies via the tree callback function. If an FSDB file contains user-defined data types, then the FSDB Reader notifies the application about user-defined data types via the tree callback function.

The FSDB Reader notifies the application about value changes via a value change traverse handle, which provides many functions to traverse the value changes back and forth. The tree callback function is usually written by the application and must have the following three parameters:

1. Callback type indicating callback data
2. Client data registered by the application using the `fprOpen()` or `fprOpen2()` call
3. Callback data

The application must perform proper type casting on callback data-based on the callback type to access right values via the typecast-structure view.

After design hierarchies are traversed in a tree callback function, the FSDB Reader does not keep the design hierarchies with a well-defined data structures in memory. Therefore, it falls to the application to keep them in memory, if necessary.

Since in FSDB, each unique variable is represented by a unique id code, it is important for the application to keep the id code of each variable. Note that id code is very important as it is the key for value change traverse. In general, the application must keep all these id codes.

The capability to contain user-defined data types is mainly for VHDL as it allows you to define enumeration type, physical type, and so on. FSDB keeps all these data-type definitions and gives each defined data type a corresponding id code, which is called data-type id code. Each variable has an associated data-type id code so that the application knows how to display the value changes, if the data type is user defined. The FSDB Reader notifies the application about the user-defined data types via a tree callback function.

For FSDB version 1.x, user-defined data types are stored and spread in design hierarchies. For version 2.x, user-defined data types are stored separately, not spread around in design hierarchies. Therefore, the application must call a specific API to read the user-defined data types. To traverse the value changes of a variable, the application must

know its id code and create a value-change traverse handle using the id code. Then, the application can traverse the value changes back and forth via traverse functions provided by the value-change traverse handle.

The application can create more than one value-change traverse handle with the same id code, thus providing different views of the value changes for the same variable. Each value-change traverse handle has an internal index, which is used to point to the current time and value change. In fact, the value-change traverse is done by increasing or decreasing this internal index.

The flow of reading FSDB is as follows:

- [Opening the FSDB File](#)
- [Traversing Design Hierarchies](#)
- [Traversing Value Changes](#)
- [Traversing User-Defined Data Types](#)
- [Selecting Variables By Their Id Codes](#)
- [Loading Value Changes for Variables Into Memory](#)
- [Creating Value-Change Traverse Handles for Variables](#)
- [Traversing Value Changes Via Traverse Handles](#)
- [Closing the FSDB File](#)

Opening the FSDB File

To open an FSDB file, use `ffrOpen` or `ffrOpen2`. The difference between `ffrOpen` and `ffrOpen2` is that `ffrOpen` activates the registered tree callback function during the call, whereas `ffrOpen2` does not activate the function. From FSDB version 2.0, the FSDB Reader supports the reading of scope hierarchies and variables. Therefore, `ffrOpen2` is mainly for the FSDB file in version 2.0 or higher, and the application can call `ffrReadScopeTree` to read scope hierarchies and `ffrReadVarByLogUOff` to read variables.

Traversing Design Hierarchies

Design hierarchies are read via the tree callback function. The application defines the tree callback function and notifies the FSDB Reader where it is by calling `ffrOpen` or `ffrOpen2` (this is called "registration"). During the call of `ffrOpen`, the FSDB Reader activates the registered tree callback function to notify the application what it has read. This is done via the two parameters: callback type and callback data. The mapping between the tree

callback type and the tree callback data can be referenced at Appendix B. During the call of `ffrOpen2`, the FSDB Reader does not activate the registered tree callback function. Instead, the application must call the specific API to activate the tree callback function, such as:

- `ffrReadScopeTree`: To know scope hierarchies.
- `ffrReadScopeVarTree`: To know both scope hierarchies and their signals.
- `ffrReadDataTypeByBlkIdx`: To know user-defined data types.

The following code snippet illustrates what we explained above:

```
bool_T MyTreeCBFunc(fsdbTreeCBType cb_type, void *client_data, void
*cb_data)
{
    switch (cb_type) {
        case FSDB_TREE_CBT_BEGIN_TREE: break;
        case FSDB_TREE_CBT_SCOPE: {
            fsdbTreeCBDataScope *scope = (fsdbTreeCBDataScope*)cb_data
            //Access scope defintion via the "scope" pointer
        }
        break;
        case FSDB_TREE_CBT_VAR: {
            fsdbTreeCBDataVar *var = (fsdbTreeCBDataVar*)cb_data;
            // access var defintion via "var" pointer
        }
        break;
        ...
    }
}

int main(int argc, char *argv[])
{
    ffrObject *obj = ffrOpen("sample.fsdb", MyTreeCBFunc, NULL);
}
```

The application defines its tree callback function as `MyTreeCBFunc` and notifies the FSDB Reader where the function is by calling `ffrOpen` (registration). During the call of `ffrOpen`, the FSDB Reader notifies the application what it has read by calling the tree callback function defined by the application, that is `MyTreeCBFunc`.

Multiple Tree Issues

Sometimes simulators have some limitations and cannot dump all signals into a single tree. Such usage might cause multiple trees in an FSDB file. For example, a Verilog dumper and a SystemC dumper in the same process may have to dump their data into the same FSDB file. In this case, there are at least two trees: one for Verilog and one for

SystemC. The tree callback function of the application takes this issue into consideration and does not assume that all signals are in a single tree.

Possible scenarios of the multiple tree usage are as follows:

- Multiple trees but scopes do not overlap.

This is the most common case. Two FSDB dumpers dump their data separately and just share the same FSDB file. Such trees are as follows:

```
BeginTree
Scope A
Signal A1
EndTree
```

```
BeginTree
Scope B
Signal B1
EndTree
```

- Multiple trees and scopes overlap.

This is also possible scenario. An FSDB dumper is unable to query all signals in a scope and needs to reenter the same scope to dump additional data later. It often happens in testbench dumping. Such trees are as follows:

```
BeginTree
Scope A
Signal A1
EndTree
```

```
BeginTree
Scope A
Signal A2
EndTree
```

- Multiple trees and scopes/signals overlap.

This should not happen. A signal should have a unique copy of the definition. However, some FSDB dumpers may go wrong and generate such FSDB files. This cannot be prevented by the FSDB writer. The FSDB Reader ignores such signals if they are detected. Such trees are as follows:

```
BeginTree
Scope A
Signal A1
EndTree
```

```
BeginTree
Scope A
Signal A1
EndTree
```

Traversing Value Changes

Traversing value changes is done via the value-change traverse class defined by the FSDB Reader. The `ffrVCIterOne` class contains numerous member functions to traverse the value changes back and forth. In the tree callback function, the application knows the id code of each variable. These id codes are important because when the application traverses the value changes of a variable, its id code is the key to create a value-change traverse object associated with it. By supplying the id code, the application calls the `ffrCreateVCTrvsHdl` API to create a traverse object and points the pointer at it, which is called "traverse handle." After that, the application can call the member functions with that traverse handle, such as `ffrGotoXTag`, `ffrGotoNextVC`, and `ffrGetVC`, to traverse the value changes back and forth. Here is the code segment.:

```
ffrObject *fsdb_obj;
fsdbVarIdcode var_idcode;
ffrVCTrvsHdl vc_trvs_hdl;
byte_T *vc;
fsdbTag64 time;
...
...
vc_trvs_hdl = fsdb_obj->ffrCreateVCTraverseHandle(var_idcode);
vc_trvs_hdl->ffrGetMinXTag((void*)&time);
vc_trvs_hdl->ffrGotoXTag(&time);

for ( ; ; ) {
    if (FSDB_RC_SUCCESS != vc_trvs_hdl->ffrGotoNextVC())
        break;

    vc_trvs_hdl->ffrGetXTag((void*)&time);
    vc_trvs_hdl->ffrGetVC((void*)&vc);
}
```

The code segment creates a value-change traverse object associated with a variable, whose id code is represented by `var_idcode`, and creates a traverse handle, `vc_trvs_hdl`. With this traverse handle, it first calls `ffrGetMinXTag` to query the minimum time where the value has changed. Then, it moves the internal index to that time by calling `ffrGotoXTag`. Finally, it calls `ffrGotoNextVC` to move the internal index forward repeatedly until there is no value change.

Traversing User-Defined Data Types

User-defined data types are traversed through a tree callback function. For FSDB version 1.x, user-defined data types and design hierarchies are stored together. However for FSDB version 2.0, user-defined data types, scopes hierarchies, and variable definitions are stored separately. As a consequence, the way to traverse user-defined data types between FSDB version 1.x and FSDB version 2.0 is different.

For FSDB version 1.x, all user-defined data types are known in the tree callback function after the tree callback function is activated. For FSDB version 2.0 or higher, the application that activates the tree callback function to notify the application all user-defined data types must call `ffrReadDataTypeDefByBlkIdx`.

- `ffrReadScopeTree()`: Notifies the application about scope hierarchies.
- `ffrReadVarByLogUOff()`: Notifies the application that all variables belonged to a specific scope.
- `ffrReadScopeVarTree()`: Notifies the application about both scope hierarchies and all variables in each scope.
- `ffrReadDataTypeDefByBlkIdx()`: Notifies the application about all user-defined data types.

To determine FSDB version 1.x and FSDB version 2.0, call `ffrHasScopeTree()`. If it returns FALSE, then the version is 1.x. If it returns TRUE, then the version is 2.0. The application can call `ffrHasDataTypeDef()` to notify whether an FSDB file contains user-defined data types.

Selecting Variables By Their Id Codes

Before loading value changes onto memory, the application must notify the FSDB Reader about variables through `ffrAddToSignalList`.

Loading Value Changes for Variables Into Memory

Once the required variables are selected, the value changes for the variables are loaded onto memory by calling `ffrLoadSignals()`. Value changes of unselected variables are not loaded onto memory.

Creating Value-Change Traverse Handles for Variables

To traverse value changes of a variable, the application must create the value-change traverse handle associated with it. Then, the value changes can be traversed back and forth via the member functions of the traverse handle. Each traverse handle has its own internal index to point to a current time and value change. The value-change traverse is done by increasing or decreasing the internal index.

The application can create more than one traverse handle associated with a variable. In that way, the application can traverse the value change at different times simultaneously via these traverse handles.

Traversing Value Changes Via Traverse Handles

The traverse handle is a pointer that points to a traverse object, and the value changes are traversed via the member functions of the traverse object.

Here are the descriptions of each member functions:

- `ffrGotoXTag`: Moves the internal index to the proper position based on the time specified by the application.
- `ffrGotoPrevVC`: Moves the internal index to the previous time and value change.
- `ffrGotoNextVC`: Moves the internal index to the next time and value change.
- `ffrGetXTag`: Returns the current time based on the internal index.
- `ffrGetVC`: Returns the current value change based on the internal index.
- `ffrGetSeqNum`: Returns the sequence number of the current value change at the current time. This API is valid only when the FSDB file contains the sequence number information.
- `ffrGetMinXTag`: Returns the minimum time where the value change occurs.
- `ffrGetMaxXTag`: Returns the maximum time where the value change occurs.
- `ffrHasIncoreVC`: Notifies the application whether this variable has value changes.
- `ffrFree`: Frees the data structures of the traverse handle. However, the value changes of the associated variable are not freed.
- `ffrGetBitSize`: Returns the bit size of the associated variable.
- `ffrGetBytesPerBit`: Returns bytes per bit of the associated variable.
- `ffrGetVarType`: Returns the variable type of the associated variable.

Closing the FSDB File

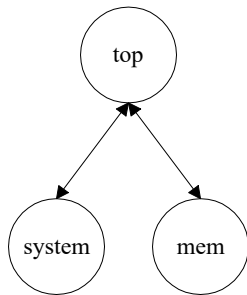
To close an FSDB file, call `ffrClose()` to perform cleanup tasks. `ffrClose()` may not actually close the FSDB file, as the FSDB Reader keeps a reference count for each FSDB file. This value is increased if the file is reopened and decreased if closed. When `ffrClose()` is called, the FSDB Reader first decreases the reference count. If, as a result, it equals to zero, then the FSDB Reader closes the FSDB file. If not, the FSDB Reader does not close the file.

Scope Tree

Scope tree is supported from FSDB version 2.0 to reduce the time it takes to traverse design hierarchies. The result is that the time to open an FSDB file and draw design hierarchies is shortened. Scope tree refers to scope hierarchies and does not include the variables in each scope. Therefore, when `ffrReadScopeTree()` is called, it notifies the application about scope hierarchies, but not about the scope of each variable.

In the tree callback function, if the callback types are begin tree, scope, up-scope, or end tree, the application can receive a logical unsigned offset from the callback data. This is the key data when the application likes to get signals for that scope. For example in Figure 1, the root scope is called `top` and has two sub-scopes named `system` and `mem`. The `top` scope has a variable named `t`, the `system` scope has a variable named `s`, and the `mem` scope has a variable named `m`.

Figure 1 Scope Tree



If `ffrReadScopeTree()` is called, callback sequences may be as follows:

- begin tree
- scope top
- scope system
- up-scope
- scope mem
- up-scope
- end tree

If `ffrReadScopeVarTree()` is called, callback sequences may be as follows:

- begin tree
- scope top
- variable t

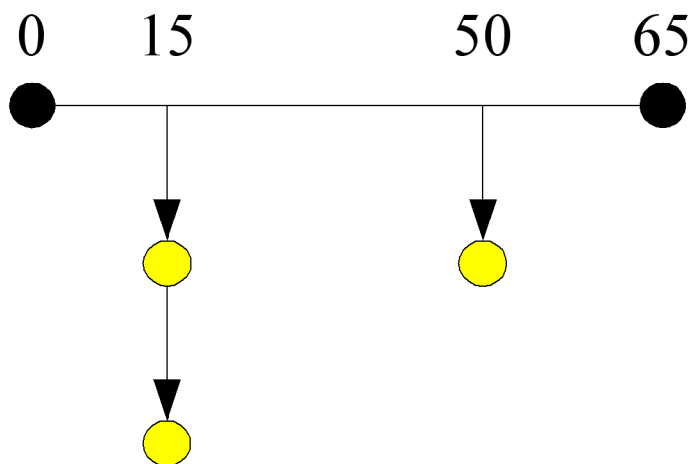
- scope system
- variable s
- up-scope
- scope mem
- variable m
- up-scope
- end tree

If the application has called `ffrReadScopeTree` and keeps the logical unsigned offset belonging to the `mem` scope, then calling `ffrReadVarByLogUOff()` with the logical unsigned offset activates the tree callback function, which notifies the application about the variable `m` (scope `mem`). This is called reading signals.

Jump Behavior

Jump behavior refers to the behavior of `ffrGotoXTag()`. The application specifies a time to which it would like to jump, but the specified time may or may not have value changes. The FSDB Reader aligns the return time, if necessary. In [Figure 2](#), the whole simulation run is from time 0 to time 65, and a variable has value change at time 15 and 50.

Figure 2 *Jump Behaviour*



If you create a value-change traverse handle associated with this variable and try to jump to a different time, the result is determined as follows:

- Jump to 5; jump fails; return time is not changed; return glitch number is 0.
- Jump to 10; jump fails; return time is not changed; return glitch number is 0.
- Jump to 15; return time is 15; return glitch number is 2.
- Jump to 65; return time is 50; return glitch number is 1.
- Jump to 30; return time is 15; return glitch number is 2.
- Jump to (-1); jump fails; return time is not changed; return glitch number is 0.
- Jump to 50; return time is 50; return glitch number is 1.
- Jump to 70; jump fails; return time is not changed; return glitch number is 0.

If the jump time has a value change, then the internal index of the traverse handle points to that time. Therefore, the return time is exactly the same as the jump time. If the jump time does not have a value change, and if the jump time is not less than the minimum time of the entire simulation run, then the return time is aligned forward. If the jump time is less than the minimum time, then the jump fails, return time is not changed and glitch number is 0. If the jump time is larger than the maximum time of the entire simulation run, then the jump fails, the return time is not changed and glitch number is 0. Note that `fFrGotoXTag()` can also query the glitch number at the return time.

Recording Dumpoff and Dumpon

Now, time range from dumpoff to dumpon is recorded into an FSDB file. Both the FSDB Writer and Reader provide APIs for the application to specify and query time range between dumpoff and dumpon. Note that dumpoff and dumpon are file based. It means when one of them is activated, it is applied onto all signals. It cannot be applied onto a single signal or partial signals. For details on reading dumpoff ranges, see `fFrObject::fFrHasDumpOffRange` and `fFrObject::fFrGetDumpOffRange`.

Getting Support Information in the FSDB File

To get support information in the FSDB file, see the description of `fFrObject::fFrHasSupportInfo` and `fFrObject::fFrGetSupportInfo`.

Traversing Value Changes in Time-Based Fashion

Traversing value changes in time-based fashion via the `ffrTimeBasedVCTrvsHdl` class. This class is different from `ffrVCTrvsHdl`, as `ffrVCTrvsHdl` is signal-based traversal and `ffrTimeBasedVCTrvsHdl` is time-based traversal.

When the application needs to traverse several signals in an FSDB file without `ffrTimeBasedVCTrvsHdl`, it has to create `ffrVCTrvsHdl` for each signal, traverse a signal from the first value change to the last one, and switch to another signal to repeat the steps above. This is called a *signal-based* value change traversal. Selected signals can be normal signals or transaction signals, but the signal type cannot be mixed. The signal type in `ffrTimeBasedVCTrvsHdl` should be the same. If `ffrTimeBasedVCTrvsHdl` has different signal types, `ffrTimeBasedVCTrvsHdl` cannot be created.

With `ffrTimeBasedVCTrvsHdl`, the application may choose to create a time-based value change traverse handle for all selected signals and traverse their chronologically ascending order. For example, suppose the value changes of signals 2, 4, 8, 9 are as follows:

	500	610	750	870
2	1		0	
4		0	1	X
8	Z		0	1
9		1	0	

If traversing them with `ffrTimeBasedVCTrvsHdl::ffrGotoNext`, the order of returned value changes appears as follows:

```
(0 500) => var(2): 1
(0 500) => var(8): Z
(0 610) => var(4): 0
(0 610) => var(9): 1
(0 750) => var(2): 0
(0 750) => var(4): 1
(0 750) => var(8): 0
(0 750) => var(9): 0
(0 870) => var(4): X
(0 870) => var(8): 1
```

Note that the sequence of value change returned at the same time step is not sorted. In the example above, at time (0 500), either the value change of signal 2 or 8 may be returned first depending on the program flow. The sequence is not guaranteed. As transactions are traversed by begin time, the transaction signal type `ffrTimeBasedVCTrvsHdl` also traverses signals by begin time.

When the application calls `ffrTimeBasedVCTrvsHdl::ffrGotoNext` to traverse, the traverse handle always stops at a certain value change of a certain signal if successful. The application may call methods of `ffrTimeBasedVCTrvsHdl`, such as

`ffrGetVarIdcode`, `ffrGetXTag`, `ffrGetVC`, and `ffrGetSeqNum`, to get the detailed value change. For the transaction signal type `ffrTimeBasedVCTrvsHdl`, the application may call transaction methods, such as `ffrGetTransInfo` and `ffrGetTransVal` to get the detailed transaction value change.

It is recommended to release the time-based value change traverse handle when finished. Otherwise, the application does not unload the value changes of the selected signals in the traverse handle as their reference count is still not 0.

Transaction Traversal

Traversing transaction values is similar to the way the value changes of normal signals are traversed. The only difference is that you should call `ffrVCIterOne::ffrGetTransVal` or `ffrVCIterOne::ffrGetTransInfo` instead of `ffrVCIterOne::ffrGetVC` to get the transaction value at certain time step. Note that the current time step aligns with begin time of the transaction value. That is, transactions are traversed by begin time.

Below is a simple program that demonstrates transactions traversal:

```
ffr_obj->ffrAddToSignalList(stream_idcode);
ffr_obj->ffrLoadSignals();

vc_trvs_hdl = ffr_obj->ffrCreateVCTraverseHandle(stream_idcode);
if (FALSE == vc_trvs_hdl->ffrHasIncoreVC()) {
    printf("This stream(%u) has no value change at all.\n",
        stream_idcode);
}
else {
    ushort_t *attr_hdl_arr;
    byte_T   **attr_val_arr;

    if (FSDB_RC_SUCCESS == vc_trvs_hdl->ffrGotoTheFirstVC()) {
        while (1) {
            vc_trvs_hdl->ffrGetTransVal(trans_id,
                label, begin_time, end_time, status, attr_count,
                attr_hdl_arr, attr_val_arr, seq_num);

            if (FSDB_RC_SUCCESS != vc_trvs_hdl->ffrGotoNextVC())
                break;
        }
    }
}
```

Although the example above uses `ffrVCIterOne::ffrGetTransVal` to get transaction values, it is recommended to use `ffrVCIterOne::ffrGetTransInfo` in future as it is easier to extend. `ffrVCIterOne::ffrGetTransInfo` is supported from FSDB version 2.3.

`ffrTransInfo` filled by `ffrVCIterOne::ffrGetTransInfo` contains the returned value of a transaction. Below is the description of each field in it:

Field Name	Description
<code>trans_id</code>	Specifies id of the transaction
<code>label</code>	Specifies label of the transaction
<code>begin_time</code>	Specifies begin time of the transaction.
<code>end_time</code>	Specifies end time of the transaction.
<code>status</code>	Specifies the status of the transaction. Its possible values are as follows: • <code>FSDB_TRANS_STATUS_UNKNOWN</code>: Shows that the transaction encounters unexpected errors. • <code>FSDB_TRANS_STATUS_PENDING</code>: Shows that the transaction is started but did not end yet. • <code>FSDB_TRANS_STATUS_FINISHED</code>: Shows that the transaction is finished successfully. • <code>FSDB_TRANS_STATUS_CANCELLED</code>: The transaction is canceled. This value is not yet supported.
<code>attr_count</code>	Specifies the number of attributes of the transaction.
<code>attr_hdl_arr</code>	Specifies an array of attribute handles. Its size is equal to <code>attr_count</code> . It is used to interpret the values of <code>attr_val_arr</code> .
<code>attr_val_arr</code>	Specifies an array of attributes value pointers. Its size is equal to <code>attr_count</code> . It is interpreted by the handles in <code>attr_hdl_arr</code> . If the i^{th} attribute in <code>attr_val_arr</code> is an array of strings, the application must treat its value as an array of string pointers that is arranged as: <pre>val[0] = "string1"; val[1] = "string2";</pre> For example, if the 2 nd attribute is an array of strings, the decoding routine may be as follows: <pre>char **str_idx_arr = (char**)attr_val_arr[1]; for (int i=0; i < num_of_elem_in_the_attr; i++) printf("%s, ", str_idx_arr[i]);</pre>
<code>relation_count</code>	Specifies the number of relations of the transaction.
<code>relation_hdl_arr</code>	Specifies an array of relation handles of the transaction. Its size is equal to <code>relation_count</code> .

Field Name	Description
related_trans_arr	Specifies an array of related transactions of the transaction. Its size is equal to <code>relation_count</code> . The transaction is related to each transaction in <code>related_trans_arr</code> by a corresponding relation in <code>relation_hdl_arr</code> .

Transaction signals can traverse value changes in time-based fashion via the `fprTimeBasedVCTrvsHdl` class. Its usage is the same as `fprVCTrvsHdl`.

Memory Traversal

Memories comprise two kinds of variables: memory range variable and memory variable. Although the variable types are different, however the value changes traversal of a memory variable is the same as the way you traverse value changes of normal signals.

A memory range variable consists of a pair of integers. It records the starting and closing index of memory variables, which have value changes at a certain timestamp. The first integer is the index of the beginning memory variable and the second is the index of the closing memory variable. For example, to define a block of memory consisting of 4 words of 8 bits each, specify the name of the memory range variable as follows:

```
my_mem[0:3]
```

Also, specify the names of the memory variables as follows:

```
my_mem[0]
my_mem[1]
my_mem[2]
my_mem[3]
```

Suppose the paired value changes of `my_mem[0:3]` at time 0 (`xtag`) are 0 and 2. It means that only `my_mem[0]` to `my_mem[2]` has value changes at time 0 while `my_mem[3]` has no value change.

Code Examples

This section describes the following code examples:

read_verilog.cpp

```
//
// Program Name: read_verilog.cpp
//
```

```
// Purpose: Demonstrate how to call FSDB reader APIs to access
// the value changes of Verilog type FSDB.
//
//
// NOVAS_FSDB is internally used in NOVAS
//
#ifdef NOVAS_FSDB
#undef NOVAS_FSDB
#endif

#include "ffrAPI.h"
#include <stdio.h>
#include <stdlib.h>

#ifndef FALSE
#define FALSE 0
#endif

#ifndef TRUE
#define TRUE 1
#endif

//
// The tree callback function, it is used to traverse design
// hierarchies.
//
static bool_T __MyTreeCB(fsdbTreeCBType cb_type,
                        void *client_data, void *tree_cb_data);

//
// dump scope definition
//
static void
__DumpScope(fsdbTreeCBDataScope *scope);

//
//dump structdefinition
//
static void
__DumpStruct(fsdbTreeCBDataStructBegin* pStruct);

//
// dump var definition
//
static void
__DumpVar(fsdbTreeCBDataVar *var);

static void
__PrintTimeValChng(ffrVCTrvsHdl vc_trvs_hdl, fsdbTag64 *time,
                  byte_T *vc_ptr);
```

```

int
main(int argc, char *argv[])
{
    if (2 != argc) {
        fprintf(stderr, "usage: rv verilog_fsdb\n");
        return FSDB_RC_FAILURE;
    }

    //
    // check the file to see if it is an FSDB file or not.
    //
    if (FALSE == ffrObject::ffrIsFSDB(argv[1])) {
        fprintf(stderr, "%s is not an FSDB file.\n", argv[1]);
        return FSDB_RC_FAILURE;
    }

    //
    // Open the FSDB file.
    //
    // From FSDB v2.0 (Debussy 5.0), there are two APIs to open a
    // FSDB file: ffrOpen() and ffrOpen2(). Both APIs take three
    // parameters, the first one is the FSDB file name, the
    // second one is a tree callback function written by
    // application, the last one is the client data that
    // application would like FSDB reader to pass it back in
    // tree callback function.
    //
    // Open an FSDB file with ffrOpen(), the tree callback
    // function will be activated many times during open session;
    // open an FSDB file with ffrOpen2(), the tree callback
    // function will not be activated during open session,
    // application has to call an API called
    // "ffrReadScopeVarTree()" to activate the tree callback
    // function.
    //
    // In tree callback function, application can tell what the
    // callback data is, based on the callback type. For example,
    // if the callback type is scope(FFR_TREE_CBT_SCOPE), then
    // application knows that it has to perform
    // (fsdbTreeCBDataScope*) type case on the callback data so
    // that it can read the scope definition.
    //
    ffrObject *fsdb_obj =
    ffrObject::ffrOpen2(argv[1], __MyTreeCB, NULL);
    if (NULL == fsdb_obj) {
        fprintf(stderr, "ffrObject::ffrOpen() failed.\n");
        exit(FSDB_RC_OBJECT_CREATION_FAILED);
    }

    if (FSDB_FT_VERILOG != fsdb_obj->ffrGetFileType()) {
        fprintf(stderr, "%s is not verilog type FSDB, just
        return.\n", argv[1]);
        fsdb_obj->ffrClose();
    }
}

```

```
        return FSDB_RC_SUCCESS;
    }

    //
    // Activate the tree callback function, read the design
    // hierarchies. Application has to perform proper type case
    // on tree callback data based on the callback type, then
    // uses the type case structure view to access the wanted
    // data.
    //
    fsdb_obj->ffrReadScopeVarTree();

    //
    // Each unique var is represented by a unique idcode in FSDB
    // file, these idcodes are positive integer and continuous
    // from the smallest to the biggest one. So the maximum
    // idcode also means that how many unique variables are there
    // in this FSDB file.
    //
    // Application can know the maximum var idcode by the
    // following API:
    //
    //ffrGetMaxVarIdcode()
    //
    fsdbVarIdcode max_var_idcode = fsdb_obj->
        ffrGetMaxVarIdcode();

    //
    // To load value changes of variables onto memory,
    // application has to tell the FSDB reader about what
    // variables it's interested in. Application selects the
    // interested variables by giving their idcodes
    // to the FSDB reader, this is done by the following API:
    //
    //ffrAddToSignalList()
    //
    int i;
    for (i = FSDB_MIN_VAR_IDCODE; i <= max_var_idcode; i++)
        fsdb_obj->ffrAddToSignalList(i);

    //
    // Load the value changes of the selected variables onto
    // memory. Value changes of unselected variables are not
    // loaded.
    //
    fsdb_obj->ffrLoadSignals();

    //
    // In order to traverse the value changes of a specific var,
    // application must create a value change traverse handle for
    // that specific variable. Once the value change traverse
    // handle is created successfully, there are lots of traverse
    // functions available to traverse the value changes backward
```

```
// and forward, or jump to a specific time, etc.
//
ffrVCTrvsHdl vc_trvs_hdl =
    fsdb_obj->ffrCreateVCTraverseHandle(max_var_idcode);
if (NULL == vc_trvs_hdl) {
    fprintf(stderr, "Failed to create a traverse
        handle(%u)\n", max_var_idcode);
    exit(FSDB_RC_OBJECT_CREATION_FAILED);
}

fsdbTag64 time;
int      glitch_num;
byte_T   *vc_ptr;

//
// Check to see if this var has value changes or not.
//
if (FALSE == vc_trvs_hdl->ffrHasIncoreVC()) {
    fprintf(stderr, "This var(%u) has no value change at
        all.\n", max_var_idcode);
}
else {
    //
    // Get the maximum time(xtag) where has value change.
    //
    if (FSDB_RC_SUCCESS !=
        vc_trvs_hdl->ffrGetMaxXTag((void*)&time)) {
        fprintf(stderr, "should not happen.\n");
        exit(FSDB_RC_FAILURE);
    }
    fprintf(stderr, "trvs hdl(%u): maximum time is
        (%u %u).\n", max_var_idcode, time.H, time.L);

    //
    // Get the minimum time (xtag) where has value change.
    //
    if (FSDB_RC_SUCCESS !=
        vc_trvs_hdl->ffrGetMinXTag((void*)&time)) {
        fprintf(stderr, "should not happen.\n");
        exit(FSDB_RC_FAILURE);
    }
    fprintf(stderr, "trvs hdl(%u): minimum time is
        (%u %u).\n", max_var_idcode, time.H, time.L);

    //
    // Jump to the specific time specified by the parameter
    // of ffrGotoXTag(). The specified time may have or have not
    // value change; if it has value change, then the return time
    // is exactly the same as the specified time; if it has not
    // value change, then the return time will be aligned forward
    // (toward smaller time direction).
    //

```

```
// There is an exception for the jump alignment: If the
// specified time is smaller than the minimum time where has
// value changes, then the return time will be aligned to the
// minimum time.
//
    if (FSDB_RC_SUCCESS != vc_trvs_hdl
        ->ffrGotoXTag((void*)&time)) {
        fprintf(stderr, "should not happen.\n");
        exit(FSDB_RC_FAILURE);
    }

//
// Get the value change.
//
    if (FSDB_RC_SUCCESS == vc_trvs_hdl->ffrGetVC(&vc_ptr))
        __PrintTimeValChng(vc_trvs_hdl, &time, vc_ptr);

//
// Value change traverse handle keeps an internal index
// which points to the current time and value change; each
// traverse API may move that internal index backward or
// forward.
//
// ffrGotoNextVC() moves the internal index backward so
// that it points to the next value change and the time
// where the next value change happened.
//
    for ( ; FSDB_RC_SUCCESS == vc_trvs_hdl->ffrGotoNextVC(); )
    {
        vc_trvs_hdl->ffrGetXTag(&time);
        vc_trvs_hdl->ffrGetVC(&vc_ptr);
        __PrintTimeValChng(vc_trvs_hdl, &time, vc_ptr);
    }

//
// ffrGotoPrevVC() moves the internal index forward so
// that it points to the previous value change and the time
// where the previous value change happened.
//
    for ( ; FSDB_RC_SUCCESS == vc_trvs_hdl->ffrGotoPrevVC(); )
    {
        vc_trvs_hdl->ffrGetXTag(&time);
        vc_trvs_hdl->ffrGetVC(&vc_ptr);
        __PrintTimeValChng(vc_trvs_hdl, &time, vc_ptr);
    }
}
//
// free this value change traverse handle
//
vc_trvs_hdl->ffrFree();

//
// We have traversed the value changes associated with the
```

```
// var whose idcode is max_var_idcode, now we are going to
// traverse the value changes of the other variables. The
// idcode of the other variables is from FSDB_MIN_VAR_IDCODE,
// which is 1, to (max_var_idcode - 1)
//
for (i = FSDB_MIN_VAR_IDCODE; i < max_var_idcode; i++) {
//
// create a value change traverse handle associated with
// current traversed var.
//
    vc_trvs_hdl = fsdb_obj->ffrCreateVCTraverseHandle(i);
    if (NULL == vc_trvs_hdl) {
        fprintf(stderr, "Failed to create a traverse
            handle(%u)\n", max_var_idcode);
        exit(FSDB_RC_OBJECT_CREATION_FAILED);
    }
    fprintf(stderr, "\n");
    fprintf(stderr, "Current traversed var idcode is %u\n",
        i);

//
// Check to see if this var has value changes or not.
//
    if (FALSE == vc_trvs_hdl->ffrHasIncoreVC()) {
        fprintf(stderr, "This var(%u) has no value change at
            all.\n", i);
        vc_trvs_hdl->ffrFree();
        continue;
    }

//
// Get the minimum time (xtag) where has value change.
//
    vc_trvs_hdl->ffrGetMinXTag((void*)&time);

//
// Jump to the minimum time (xtag).
//
    vc_trvs_hdl->ffrGotoXTag((void*)&time);

//
// Traverse all the value changes from the minimum time
// to the maximum time.
// do {
    vc_trvs_hdl->ffrGetXTag(&time);
    vc_trvs_hdl->ffrGetVC(&vc_ptr);
    __PrintTimeValChng(vc_trvs_hdl, &time, vc_ptr);
} while(FSDB_RC_SUCCESS == vc_trvs_hdl->ffrGotoNextVC());

//
// free this value change traverse handle
//
    vc_trvs_hdl->ffrFree();
}
```

```

    }

    //
    // close this FSDB file.
    //
    fsdb_obj->ffrClose();
    return 0;
}

static void
__PrintTimeValChng(ffrVCTrvsHdl vc_trvs_hdl,
                  fsdbTag64 *time, byte_T *vc_ptr)
{
    static byte_T buffer[FSDB_MAX_BIT_SIZE+1];
    byte_T *ret_vc;
    uint_T i;
    fsdbVarType var_type;

    switch (vc_trvs_hdl->ffrGetBytesPerBit()) {
    case FSDB_BYTES_PER_BIT_1B:

        //
        // Convert each Verilog bit type to corresponding
        // character.
        //
        for (i = 0; i < vc_trvs_hdl->ffrGetBitSize(); i++) {
            switch(vc_ptr[i]) {
            case FSDB_BT_VCD_0:
                buffer[i] = '0';
                break;

            case FSDB_BT_VCD_1:
                buffer[i] = '1';
                break;

            case FSDB_BT_VCD_X:
                buffer[i] = 'x';
                break;

            case FSDB_BT_VCD_Z:
                buffer[i] = 'z';
                break;

            default:
                //
                // unknown Verilog bit type found.
                //
                buffer[i] = 'u';
            }
        }
        buffer[i] = '\0';
        fprintf(stderr, "time: (%u %u)  val chg: %s\n",
                time->H, time->L, buffer);
    }
}

```

```

        break;

case FSDB_BYTES_PER_BIT_4B:
    //
    // Not 0, 1, x, z since their bytes per bit is
    // FSDB_BYTES_PER_BIT_1B.
    //
    // For Verilog type FSDB, there is no array of
    // real/float/double so far, so we don't have to
    // care about that kind of case.
    //

    //
    // The var type of memory range variable is
    // FSDB_VT_VCD_MEMORY_DEPTH. This kind of var
    // has two value changes at a certain time step.
    // The first value change is the index of the
    // beginning memory variable which has a value change
    // and the second is the index of the end memory
    // variable which has a value change at this time step.
    //

var_type = vc_trvs_hdl->ffrGetVarType();
switch(var_type){
    case FSDB_VT_VCD_MEMORY_DEPTH:
        FSDB_VT_VHDL_MEMORY_DEPTH:
            fprintf(stderr, "time: (%u %u)",
                time->H, time->L);
            fprintf(stderr, "  begin: %d", *((int*)vc_ptr));
            vc_ptr = vc_ptr + sizeof(int);
            fprintf(stderr, "  end: %d\n", *((int*)vc_ptr));
            break;

    default:
        vc_trvs_hdl->ffrGetVC(&vc_ptr);
        fprintf(stderr, "time: (%u %u)  val chg: %f\n",
            time->H, time->L, *((float*)vc_ptr));
        break;
}
break;

    case FSDB_BYTES_PER_BIT_8B:
    //
    // Not 0, 1, x, z since their bytes per bit is
    // FSDB_BYTES_PER_BIT_1B.
    //
    // For Verilog type FSDB, there is no array of
    // real/float/double so far, so we don't have to
    // care about that kind of case.
    //

    fprintf(stderr, "time: (%u %u)  val chg: %e\n",
        time->H, time->L, *((double*)vc_ptr));
    break;

```

```
        default:
            fprintf(stderr, "Control flow should not reach
                           here.\n");
            break;
    }
}

static bool_T __MyTreeCB(fsdbTreeCBType cb_type,
                        void *client_data, void *tree_cb_data)
{
    switch (cb_type) {
    case FSDB_TREE_CBT_BEGIN_TREE:
        fprintf(stderr, "<BeginTree>\n");
        break;

    case FSDB_TREE_CBT_SCOPE:
        __DumpScope((fsdbTreeCBDataScope*)tree_cb_data);
        break;

    case FSDB_TREE_CBT_VAR:
        __DumpVar((fsdbTreeCBDataVar*)tree_cb_data);
        break;

    case FSDB_TREE_CBT_UPSCOPE:
        fprintf(stderr, "<Upscope>\n");
        break;

    case FSDB_TREE_CBT_END_TREE:
        fprintf(stderr, "<EndTree>\n\n");
        break;

    case FSDB_TREE_CBT_FILE_TYPE:
        break;

    case FSDB_TREE_CBT_SIMULATOR_VERSION:
        break;

    case FSDB_TREE_CBT_SIMULATION_DATE:
        break;

    case FSDB_TREE_CBT_X_AXIS_SCALE:
        break;

    case FSDB_TREE_CBT_END_ALL_TREE:
        break;

    case FSDB_TREE_CBT_ARRAY_BEGIN:
        fprintf(stderr, "<BeginArray>\n");
        break;

    case FSDB_TREE_CBT_ARRAY_END:
        fprintf(stderr, "<EndArray>\n\n");
    }
```

```
        break;

    case FSDB_TREE_CBT_RECORD_BEGIN:
        fprintf(stderr, "<BeginRecord>\n");
        break;

    case FSDB_TREE_CBT_RECORD_END:
        fprintf(stderr, "<EndRecord>\n\n");
        break;

    case FSDB_TREE_CBT_STRUCT_BEGIN:
        fprintf(stderr, "<BeginStruct>\n");
        __DumpStruct((fsdbTreeCBDataStructBegin*) tree_cb_data);
        break;

    case FSDB_TREE_CBT_STRUCT_END:
        fprintf(stderr, "<EndStruct>\n\n");
        break;

    default:
        return FALSE;
    }

    return TRUE;
}

static void
__DumpScope(fsdbTreeCBDataScope* scope)
{
    str_T type;

    switch (scope->type) {
    case FSDB_ST_VCD_MODULE:
        type = (str_T) "module";
        break;

    case FSDB_ST_VCD_TASK:
        type = (str_T) "task";
        break;

    case FSDB_ST_VCD_FUNCTION:
        type = (str_T) "function";
        break;

    case FSDB_ST_VCD_BEGIN:
        type = (str_T) "begin";
        break;

    case FSDB_ST_VCD_FORK:
        type = (str_T) "fork";
        break;

    case FSDB_ST_VCD_LIST:
```

```
        type = (str_T) "list";
        break;

    case FSDB_ST_VCD_ARRAY_ELEM:
        type = (str_T) "array elem";
        break;

    case FSDB_ST_VCD_GENERATE:
        type = (str_T) "generate";
        break;

    case FSDB_ST_VHDL_ARCHITECTURE:
        type = (str_T) "vhdl architecture";
        break;

    case FSDB_ST_VHDL_PROCEDURE:
        type = (str_T) "vhdl procedure";
        break;

    case FSDB_ST_VHDL_FUNCTION:
        type = (str_T) "vhdl function";
        break;

    case FSDB_ST_VHDL_FUNCTION:
        type = (str_T) "vhdl function";
        break;

    case FSDB_ST_VHDL_RECORD:
        type = (str_T) "vhdl record";
        break;

    case FSDB_ST_VHDL_PROCESS:
        type = (str_T) "vhdl process";
        break;

    case FSDB_ST_VHDL_PROCESS:
        type = (str_T) "vhdl process";
        break;

    case FSDB_ST_VHDL_BLOCK:
        type = (str_T) "vhdl block";
        break;

    case FSDB_ST_VHDL_FOR_GENERATE:
        type = (str_T) "vhdl for generate";
        break;

    case FSDB_ST_VHDL_IF_GENERATE:
        type = (str_T) "vhdl if generate";
        break;

    case FSDB_ST_VHDL_GENERATE:
        type = (str_T) "vhdl generate";
```

```

        break;

    case FSDB_ST_SV_INTERFACE:
        type = (str_T) "system Verilog interface";
        break;

    default:
        type = "unknown_scope_type";
        break;
}

fprintf(stderr, "<Scope> name:%s  type:%s\n",
        scope->name, type);
}

static void
__DumpStruct(fsdbTreeCBDataStructBegin* pStruct)
{
    str_T type;
    switch(pStruct->type) {
    case FSDB_STRUCT_TYPE_VHDL_RECORD:
        type = (str_T) "vhdl_record";
        break;

    case FSDB_STRUCT_TYPE_HVL_DL:
        type = (str_T) "hvl_driver_load";
        break;

    case FSDB_STRUCT_TYPE_UNPACKED_STRUCT:
        type = (str_T) "unpacked_struct";
        break;

    case FSDB_STRUCT_TYPE_PACKED_STRUCT:
        type = (str_T) "packed_struct";
        break;
    case FSDB_STRUCT_TYPE_UNPACKED_UNION:
        type = (str_T) "unpacked_union";
        break;

    case FSDB_STRUCT_TYPE_PACKED_UNION:
        type = (str_T) "packed_union";
        break;

    default:
        type = "unknown struct type";
        break;
    }

    fprintf(stderr, "Struct Name: %s, type: %s, Field Count:
        %u\n", pStruct->name, type, pStruct->fieldCount);
}

```

```
static void
DumpVar(fsdbTreeCBDataVar *var)
{
    str_T type;
    str_T bpb;

    switch(var->bytes_per_bit) {
    case FSDB_BYTES_PER_BIT_1B:
        bpb = (str_T) "1B";
        break;

    case FSDB_BYTES_PER_BIT_2B:
        bpb = (str_T) "2B";
        break;

    case FSDB_BYTES_PER_BIT_4B:
        bpb = (str_T) "4B";
        break;

    case FSDB_BYTES_PER_BIT_8B:
        bpb = (str_T) "8B";
        break;

    default:
        bpb = (str_T) "xB";
        break;
    }

    switch (var->type) {
    case FSDB_VT_VCD_EVENT:
        type = (str_T) "event";
        break;

    case FSDB_VT_VCD_INTEGER:
        type = (str_T) "integer";
        break;

    case FSDB_VT_VCD_PARAMETER:
        type = (str_T) "parameter";
        break;

    case FSDB_VT_VCD_REAL:
        type = (str_T) "real";
        break;

    case FSDB_VT_VCD_REG:
        type = (str_T) "reg";
        break;

    case FSDB_VT_VCD_SUPPLY0:
        type = (str_T) "supply0";
        break;
    }
```

```
case FSDB_VT_VCD_SUPPLY1:
    type = (str_T) "supply1";
    break;

case FSDB_VT_VCD_TIME:
    type = (str_T) "time";
    break;

case FSDB_VT_VCD_TRI:
    type = (str_T) "tri";
    break;

case FSDB_VT_VCD_TRIAND:
    type = (str_T) "triand";
    break;

case FSDB_VT_VCD_TRIOR:
    type = (str_T) "trior";
    break;

case FSDB_VT_VCD_TRIREG:
    type = (str_T) "triereg";
    break;

case FSDB_VT_VCD_TRI0:
    type = (str_T) "tri0";
    break;

case FSDB_VT_VCD_TRI1:
    type = (str_T) "tri1";
    break;

case FSDB_VT_VCD_WAND:
    type = (str_T) "wand";
    break;

case FSDB_VT_VCD_WIRE:
    type = (str_T) "wire";
    break;

case FSDB_VT_VCD_WOR:
    type = (str_T) "wor";
    break;

case FSDB_VT_VHDL_SIGNAL:
    type = (str_T) "signal";
    break;

case FSDB_VT_VHDL_VARIABLE:
    type = (str_T) "variable";
    break;

case FSDB_VT_VHDL_CONSTANT:
```

```

        type = (str_T) "constant";
        break;

    case FSDB_VT_VHDL_FILE:
        type = (str_T) "file";
        break;

    case FSDB_VT_VCD_MEMORY:
        type = (str_T) "vcd_memory";
        break;

    case FSDB_VT_VHDL_MEMORY:
        type = (str_T) "vhdl_memory";
        break;

    case FSDB_VT_VCD_MEMORY_DEPTH:
        type = (str_T) "vcd_memory_depth ";
        break;

    case FSDB_VT_VHDL_MEMORY_DEPTH:
        type = (str_T) "vhdl_memory_depth";
        break;

    default:
        type = (str_T) "unknown_var_type";
        break;
    }

    fprintf(stderr,
        "<Var> name:%s l:%u r:%u type:%s ",
        var->name, var->lbitnum, var->rbitnum, type);
    fprintf(stderr,
        "idcode:%u dtidcode:%u bpb:%s\n",
        var->u.idcode, var->dtidcode, bpb);
}

```

tb.cpp

```

//
// Program Name : tb.cpp
//
// Purpose : Demonstrate how to traverse signal in Time-based fashion
//
//
// NOVAS_FSDB is internally used in NOVAS
//

#ifdef NOVAS_FSDB
#undef NOVAS_FSDB
#endif

```

```
#include "stdio.h"
#include "stdlib.h"
#include "string.h"
#include "stdarg.h"
#include "assert.h"
#include "ffwAPI.h"
#include "ffrAPI.h"

//
// Constants
//
#ifndef TRUE
    const int TRUE = 1;
#endif

#ifndef FALSE
    const int FALSE = 0;
#endif

const int          SIG_NUM          = 7;

//
// Global variable definition and declaration
//
struct ScalarSignal {
    char            *name;           // signal name
    fsdbVarType     type;            // signal type
    void*           dt;              // signal data type
    ushort_T        lbitnum;         // signal left bit number
    fsdbVarDir       direction;      // signal direction
    ushort_T        rbitnum;         // signal right bit number
    fsdbBytesPerBit bpb;             // signal bytes per bit
};
typedef struct ScalarSignal ScalarSignal;

static ScalarSignal sig1 = {
    "sig1", FSDB_VT_VCD_WIRE, FSDB_DT_HDL_VERILOG_STANDARD,
    0, 0, FSDB_VD_IMPLICIT, FSDB_BYTES_PER_BIT_1B
};

static ScalarSignal sig2 = {
    "sig2", FSDB_VT_VCD_WIRE, FSDB_DT_HDL_VERILOG_STANDARD,
    0, 0, FSDB_VD_IMPLICIT, FSDB_BYTES_PER_BIT_1B
};

static ScalarSignal sig3 = {
    "sig3", FSDB_VT_VCD_WIRE, FSDB_DT_HDL_VERILOG_STANDARD,
    0, 0, FSDB_VD_IMPLICIT, FSDB_BYTES_PER_BIT_1B
};

static ScalarSignal sig4 = {
    "sig4", FSDB_VT_VCD_WIRE, FSDB_DT_HDL_VERILOG_STANDARD,
    0, 0, FSDB_VD_IMPLICIT, FSDB_BYTES_PER_BIT_1B
};
```

```
};

static ScalarSignal sig5 = {
    "sig5", FSDB_VT_VCD_WIRE, FSDB_DT_HDL_VERILOG_STANDARD,
    0, 0, FSDB_VD_IMPLICIT, FSDB_BYTES_PER_BIT_1B
};

static ScalarSignal sig6 = {
    "sig6", FSDB_VT_VCD_WIRE, FSDB_DT_HDL_VERILOG_STANDARD,
    0, 0, FSDB_VD_IMPLICIT, FSDB_BYTES_PER_BIT_1B
};

static ScalarSignal sig7 = {
    "sig7", FSDB_VT_VCD_WIRE, FSDB_DT_HDL_VERILOG_STANDARD,
    0, 0, FSDB_VD_IMPLICIT, FSDB_BYTES_PER_BIT_1B
};

static fsdbRC
__TBTraversal(void);

static void
__CreateTree(ffwObject* ffw_obj);

static void
__CreateVC(ffwObject* ffw_obj);

static fsdbRC
__TraverseVC(ffrObject* ffr_obj);

static void
__OpenFsdbFileForWrite(ffwObject **fsdb_obj, char fname[],
    fsdbFileType file_type);

static fsdbRC
__OpenFsdbFileForRead(ffrObject **fsdb_obj, char fname[]);

static void
PrintAsVerilog(byte_T *ptr, uint_T size);

//
// Main Program
//
int
main(int argc, char *argv[])
{
    __TBTraversal();

    return FSDB_RC_SUCCESS;
}

//
// __TBTraversal()
//
```

```
fsdbRC
__TBTraversal(void)
{
    ffwObject    *ffw_obj = NULL;
    ffrObject    *ffr_obj = NULL;
    char         fname[] = "tb_trvs.fsdb";
    fsdbRC       rc = FSDB_RC_SUCCESS;
    fsdbXTag     start_xtag, close_xtag;

    //
    // Write to the FSDB file
    //
    __OpenFsdbFileForWrite(&ffw_obj, fname, FSDB_FT_VERILOG);
    ffw_SetGlitchNum(ffw_obj, 0);

    //
    // Turn on sequence number if necessary. Sequence number will affect
the
    // order of value change returned.
    //
    ffw_DumpSeqNumTurnOn(ffw_obj);

    __CreateTree(ffw_obj);
    __CreateVC(ffw_obj);
    ffw_Close(ffw_obj);

    //
    // Open the FSDB file for read
    //
    rc = __OpenFsdbFileForRead(&ffr_obj, fname);
    if (rc != FSDB_RC_SUCCESS) {
        fprintf(stderr, "Fsdb file(%s) failed to be opened for read.\n",
fname);
        assert(FALSE);
    }

    //
    // Set view window if necessary. Only value changes within the view
window
    // will be returned.
    //
    start_xtag.hltag.H = 0;
    start_xtag.hltag.L = 600;
    close_xtag.hltag.H = 0;
    close_xtag.hltag.L = 800;
    ffr_obj->ffrSetViewWindow(&start_xtag, &close_xtag);

    //
    // Traverse the vc in the FSDB file
    //
    rc = __TraverseVC(ffr_obj);
    fprintf(stdout, "Time-based traversal done!!.\n");
}
```

```

        ffr_obj->ffrClose();

    return rc;
}

//
// __CreateTree()
//
void
__CreateTree(ffwObject* ffw_obj)
{
    char            name[100];
    ffwVarMapId     var_map_id;

    fprintf(stdout, "==== Creating tree =====\n");

    // Create scope 'Top'
    ffw_BeginTree(ffw_obj);
    ffw_CreateScope(ffw_obj, FSDB_ST_VCD_MODULE, "Top");

    var_map_id = ffw_CreateVarByHandle(ffw_obj, sig1.type,
                                       sig1.direction, sig1.dt, sig1.lbitnum, sig1.rbitnum,
                                       &sig1, sig1.name, sig1.bpb);

    var_map_id = ffw_CreateVarByHandle(ffw_obj, sig2.type,
                                       sig2.direction, sig2.dt, sig2.lbitnum, sig2.rbitnum,
                                       &sig2, sig2.name, sig2.bpb);

    var_map_id = ffw_CreateVarByHandle(ffw_obj, sig3.type,
                                       sig3.direction, sig3.dt, sig3.lbitnum, sig3.rbitnum,
                                       &sig3, sig3.name, sig3.bpb);

    var_map_id = ffw_CreateVarByHandle(ffw_obj, sig4.type,
                                       sig4.direction, sig4.dt, sig4.lbitnum, sig4.rbitnum,
                                       &sig4, sig4.name, sig4.bpb);

    var_map_id = ffw_CreateVarByHandle(ffw_obj, sig5.type,
                                       sig5.direction, sig5.dt, sig5.lbitnum, sig5.rbitnum,
                                       &sig5, sig5.name, sig5.bpb);

    var_map_id = ffw_CreateVarByHandle(ffw_obj, sig6.type,
                                       sig6.direction, sig6.dt, sig6.lbitnum, sig6.rbitnum,
                                       &sig6, sig6.name, sig6.bpb);

    var_map_id = ffw_CreateVarByHandle(ffw_obj, sig7.type,
                                       sig7.direction, sig7.dt, sig7.lbitnum, sig7.rbitnum,
                                       &sig7, sig7.name, sig7.bpb);

    // Tree created
    ffw_EndTree(ffw_obj);
    fprintf(stdout, "Tree created.\n");
}

```

```
//
// __CreateVC()
//
void
__CreateVC(ffwObject* ffw_obj)
{
    static byte_T    val;
    int              sig_idx, xtag_idx;
    fsdbTag64        time;

    //
    //      | 500 | 610 | 750 | 870
    // -----
    //  1 |  10 |    |  x1 |
    //  2 |    |  0 | 1x0 |  1
    //  3 |  Z  |    |  0 |  1
    //  4 |    |  1 |  0 |
    //  5 |  1  |    |    |  x
    //  6 |    |    |    |  z
    //  7 |    |    |    |
    //

    time.H = 0;
    time.L = 500;
    ffw_CreateXCoorByHnL(ffw_obj, time.H, time.L);
    fprintf(stdout, "Create value change in (%d, %d).\n", time.H,
time.L);

    val = FSDB_BT_VCD_1;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig1, &val);
    val = FSDB_BT_VCD_0;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig1, &val);
    val = FSDB_BT_VCD_Z;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig3, &val);
    val = FSDB_BT_VCD_1;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig5, &val);

    time.H = 0;
    time.L = 610;
    ffw_CreateXCoorByHnL(ffw_obj, time.H, time.L);
    fprintf(stdout, "Create value change in (%d, %d).\n", time.H,
time.L);

    val = FSDB_BT_VCD_0;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig2, &val);
    val = FSDB_BT_VCD_1;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig4, &val);

    time.H = 0;
    time.L = 750;
    ffw_CreateXCoorByHnL(ffw_obj, time.H, time.L);
    fprintf(stdout, "Create value change in (%d, %d).\n", time.H,
time.L);
}
```

```

    val = FSDB_BT_VCD_X;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig1, &val);
    val = FSDB_BT_VCD_1;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig1, &val);
    val = FSDB_BT_VCD_1;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig2, &val);
    val = FSDB_BT_VCD_X;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig2, &val);
    val = FSDB_BT_VCD_0;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig2, &val);
    val = FSDB_BT_VCD_0;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig3, &val);
    val = FSDB_BT_VCD_0;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig4, &val);

    time.H = 0;
    time.L = 870;
    ffw_CreateXCoorByHnL(ffw_obj, time.H, time.L);
    fprintf(stdout, "Create value change in (%d, %d).\n", time.H,
    time.L);

    val = FSDB_BT_VCD_1;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig2, &val);
    val = FSDB_BT_VCD_1;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig3, &val);
    val = FSDB_BT_VCD_X;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig5, &val);
    val = FSDB_BT_VCD_Z;
    ffw_CreateVarValueByHandle(ffw_obj, (void*) &sig6, &val);
}

static fsdbRC
__TraverseVC(ffrObject* ffr_obj)
{
    fsdbVarIdcode      sig_arr[SIG_NUM] = {1, 2, 3, 4, 5, 6, 7};
    ffrTimeBasedVCTrvsHdl  tb_vc_trvs_hdl;
    ffrTimeBasedVCTrvsHdl  tb_vc_trvs_hdl1;
    ffrTimeBasedVCTrvsHdl  tb_vc_trvs_hdl2;
    ffrTimeBasedVCTrvsHdl  tb_vc_trvs_hdl3;
    byte_T                *vc_ptr;
    fsdbVarIdcode          var_idcode;
    fsdbXTag                time;
    fsdbSeqNum              seq_num;
    fsdbRC                  rc = FSDB_RC_SUCCESS;
    int                     sig_idx;

    // Read all vars
    ffr_obj->ffrReadScopeVarTree();

    // Load value change
    for (sig_idx = 0; sig_idx < SIG_NUM; sig_idx++)
        ffr_obj->ffrAddToSignalList(sig_arr[sig_idx]);

```

```

    ffr_obj->ffrLoadSignals();

    //
    // Create a time-based traverse handle to encapsulate the signals to
be
    // traversed.
    //
    tb_vc_trvs_hdl = ffr_obj->
        ffrCreateTimeBasedVCTrvsHdl(SIG_NUM, sig_arr);

    if (NULL == tb_vc_trvs_hdl) {
        fprintf(stderr, "Fail to create time-based value change
            trvs
            hdl!\n");
        return FSDB_RC_FAILURE;
    }

    if (FSDB_RC_SUCCESS == tb_vc_trvs_hdl->ffrGetVC(&vc_ptr)) {
        tb_vc_trvs_hdl->ffrGetVarIdcode(&var_idcode);
        tb_vc_trvs_hdl->ffrGetXTag(&time);
        tb_vc_trvs_hdl->ffrGetSeqNum(&seq_num);

        fprintf(stdout, "(%u %u) => var(%u): seq_num: %u val: ",
            time.hltag.H, time.hltag.L, var_idcode, seq_num);
        PrintAsVerilog(vc_ptr, 1);
    }

    //
    // Iterate until no more value change
    //
    // The order of the value change at the same time step are
    // returned according to their sequence number. If sequence
    // number is not turned on, the order will be undetermined.
    //
    while (FSDB_RC_SUCCESS == tb_vc_trvs_hdl->ffrGotoNextVC()) {
        tb_vc_trvs_hdl->ffrGetVarIdcode(&var_idcode);
        tb_vc_trvs_hdl->ffrGetXTag(&time);
        tb_vc_trvs_hdl->ffrGetSeqNum(&seq_num);
        tb_vc_trvs_hdl->ffrGetVC(&vc_ptr);

        fprintf(stdout, "(%u %u) => var(%u): seq_num: %u val: ",
            time.hltag.H, time.hltag.L, var_idcode, seq_num);
        PrintAsVerilog(vc_ptr, 1);
    }

    //
    // Remember to call ffrFree() to free the memory occupied by
    // this time-based value change trvs hdl
    //
    tb_vc_trvs_hdl->ffrFree();

    //

```

```

        // Remember to call ffrUnloadSignals() to unload value change      //
memory.
    //
    ffr_obj->ffrUnloadSignals();

    return rc;
}

/*
** Print as Verilog standard values.
** x0z1-z10z 0x0x-101l...
*/
void
PrintAsVerilog(byte_T *ptr, uint_T size)
{
    const int VALUES_IN_A_SEG      = 8;
    const int VALUES_DUMPED_IN_A_LINE = 40;
    int i, j, end_idx;
    byte_T a_byte;
    char val_tbl[] = "01xz";

    for(i = 0; i < size; i++) {
        a_byte = *(ptr+i);

        if (a_byte < 4)
            fprintf(stdout, "%c", val_tbl[a_byte]);
        else
            fprintf(stdout, "?");

        if (3 == (i % VALUES_IN_A_SEG) && (i < size-1))
            fprintf(stdout, "-");

        if ((VALUES_IN_A_SEG - 1) == (i % VALUES_IN_A_SEG)) {
            if ((VALUES_DUMPED_IN_A_LINE - 1) ==
                (i % VALUES_DUMPED_IN_A_LINE)) {
                fprintf(stdout, "\n");
            }
            else {
                fprintf(stdout, " ");
            }
        }
    }

    if ((VALUES_DUMPED_IN_A_LINE - 1) != (i % VALUES_DUMPED_IN_A_LINE)) {
        fprintf(stdout, "\n");
    }
}

//
// OpenFsdbFileForWrite()
//
void
__OpenFsdbFileForWrite(ffwObject **fsdb_obj, char fname[],

```

```
fsdbFileType file_type)
{
    // Open fsdb file
    *fsdb_obj = ffw_Open(fname, file_type);
    if (NULL == *fsdb_obj) {
        fprintf(stderr, "Fail to open a fast file object! File
            name = %s.\n", fname);
        exit(FSDB_RC_OBJECT_CREATION_FAILED);
    }

    // Use handle scheme for tree creation
    ffw_CreateTreeByHandleScheme(*fsdb_obj);
    ffw_SetScaleUnit(*fsdb_obj, "0.01p");
}

//
// OpenFsdbFileForRead()
//
fsdbRC
__OpenFsdbFileForRead(ffrObject **fsdb_obj, char fname[])
{
    //
    // Check and open FSDB file
    //
    if (FALSE == ffrObject::ffrIsFSDB(fname)) {
        fprintf(stderr, "%s is not an FSDB file.\n", fname);
        return FSDB_RC_FAILURE;
    }

    *fsdb_obj = ffrObject::ffrOpen2(fname, NULL, NULL);

    if (NULL == *fsdb_obj) {
        fprintf(stderr, "ffrObject::ffrOpen2() failed(%s).\n",
            fname);
        return FSDB_RC_FAILURE;
    }

    return FSDB_RC_SUCCESS;
}
```

2

Function List

This section describes the following topics:

- [ffrObject](#)
- [ffrTimeBasedVCIterOne](#)
- [ffrVCIterOne](#)
- [Other](#)

ffrObject

ffrAddToSignalList

Interface

```
fsdbRC ffrAddToSignalList(fsdbVarIdcode idcode, ffrMinMaxAnalogValRec
*min_max_analog_val = NULL, str_T full_path_var_name = NULL);
```

Purpose

Selects a signal of interest into a signal list. The chosen scheme is based on variable `idcode` because it identifies a unique signal.

Arguments

`idcode`

The idcode of the signal of interest.

`*min_max_analog_val`

A pointer to min max analog value record. The space is allocated by caller and its content is filled by the FSDB Reader. It has a default value and is used by *nWave* only. The application can just ignore it.

`full_path_var_name`

A variable name with full path. This argument is the only way for OEM to know the variable information, because OEM does not register a tree callback function. This argument has a default value and is used only by *nWave*. The application can just ignore it.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

ffrCheckFile

Interface

```
static fsdbRC ffrObject::ffrCheckFile (str_T file_name);
```

Purpose

Checks the file to see if it is a regular and readable file or not.

Arguments

file_name

A pointer to a string which represents the file name.

Return Value

FSDB_RC_SUCCESS if the file is regular and readable.

FSDB_RC_FILE_DOES_NOT_EXIST if file does not exist.

FSDB_RC_FILE_IS_NOT_READABLE if file is not readable.

FSDB_RC_FILE_IS_A_DIRECTORY if file is a directory.

See Also

ffrObject::ffrCheckFSDB

ffrObject::ffrIsFSDB

ffrCheckFSDB

Interface

```
static fsdbRC ffrObject::ffrCheckFSDB (str_T fsdb_fname);
```

Purpose

Checks if it is a regular and readable FSDB type. This API combines **ffrCheckFile** and **ffrIsFSDB**.

Arguments

`fsdb_fname`

A pointer to a string which represents the FSDB file name.

Return Value

FSDB_RC_SUCCESS if the file is regular, readable and of FSDB type.

FSDB_RC_FAILURE if one of these tree conditions is not true.

See Also

`ffrObject::ffrCheckFile`

`ffrObject::ffrIsFSDB`

ffrClose

Interface

```
void ffrObject::ffrClose();
```

Purpose

Closes an FSDB file, performs necessary clean up jobs and frees memory.

See Also

`ffrObject::ffrOpen`

`ffrObject::ffrOpen2`

`ffrObject::ffrOpen3`

ffrCreateTimeBasedVCTrvsHdl

Interface

```
ffrTimeBasedVCTrvsHdl ffrObject::ffrCreateTimeBasedVCTrvsHdl (uint_T num,  
fsdbVarIdcode sig_arr[]);
```

Purpose

Creates a traverse handle associated with the signals which are specified in *sig_arr[]*. The number of signals in *sig_arr[]* is specified by *num*. The signal types in *sig_arr[]* must be the same. If signal types are different, the traverse handle cannot be created. The application can traverse the value changes of these signal in a time-based fashion through

this traverse handle. See **Traverse Value Changes in Time-based fashion** section for details.

Arguments

`num`

The number of signals in `sig_arr[]`.

`sig_arr`

The idcodes of the signals that are to be traversed.

Return Value

Non-null if successful; null otherwise.

See Also

`ffrObject::ffrCreateVCTrvsHdl`

ffrCreateVCTrvsHdl

Interface

```
ffrVCTrvsHdl ffrObject::ffrCreateVCTrvsHdl (fsdbVarIdcode var_idcode);
```

Purpose

Creates a traverse handle associated with the signal which is specified by its idcode. The application can traverse the value changes of a signal through its traverse handle. If the value changes of a signal need to be traversed in different ways at the same time, the application can associate multiple traverse handles with the signal.

Arguments

`var_idcode`

The idcode of the signal that is to be traversed.

Return Value

Non-null if successful; null otherwise.

ffrExtractScaleUnit

Interface

```
static fsdbRC ffrObject::ffrExtractScaleUnit (str_T scale_unit, uint_T &digit, char *&unit);
```

Purpose

Extract the digit and unit part in a scale unit string. For example, if the argument *scale_unit* is 300ps, *digit* is set to 300 and *unit* is set to ps.

Arguments

scale_unit

The scale unit string that is to be extracted.

digit

The extracted digit part of scale unit.

**nit*

The extracted unit part of scale unit.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffw_SetScaleUnit

ffrObject::ffrGetScaleUnit

ffrFreeNavDB

Interface

fsdbRC ffrObject::ffrFreeNavDB()

Purpose

Free all NavDB that are used by signals in the signal list.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

ffrGetAllDumperEnvVarVal;

Interface

fsdbRC ffrObject::ffrGetAllDumperEnvVarVal(str_T all);

Purpose

Get all dumper environment variables and their values in the FSDB file. The input strings are allocated by caller. This API is used from FSDB version 2.0.

Arguments

`all`

All dumper environment variable names.

Return Value

FSDB_RC_SUCCESS on success; **FSDB_RC_FAILURE** otherwise.

See Also

`ffrObject::ffrGetDumperEnvVarVal`

ffrGetAnnotation

Interface

`str_T ffrObject::ffrGetAnnotation (uint_T &len) const;`

Purpose

Get annotation if any FSDB allows the application to write annotation into FSDB file. The annotation is not written to FSDB file until it is closed. The maximum length of the annotation is defined as **FSDB_ANNOTATION_LEN** in the **fsdbShr.h** header file.

Arguments

`len`

The length of the annotation stored in FSDB file. `len` is always smaller than or equal to **FSDB_ANNOTATION_LEN**.

Return Value

Non-null if the FSDB contains annotation; null if it does not.

See Also

`ffw_SetAnnotation`

ffrGetAPIVersion

Interface

```
static str_T ffrObject::ffrGetAPIVersion();
```

Purpose

Get the FSDB Reader API version.

Return Value

The version string of the FSDB Reader API.

ffrGetDumperEnvVarVal;

Interface

```
Str_T ffrObject::ffrGetDumperEnvVarVal(str_T env_var);
```

Purpose

Retrieve the dumper environment variables in the FSDB file. This API is used from FSDB version 2.0.

Arguments

env_var

Specify the name of environment variables.

Return Value

The value of specific environment variable; null if environment has no specific variable

See Also

ffrObject::ffrGetAllDumperEnvVarVal

ffrGetDumpOffRange

Interface

```
fsdbRC ffrObject::ffrGetDumpOffRange (uint_T &count, fsdbDumpOffRange *&range);
```

Purpose

Retrieve the dump off ranges in the FSDB file. An FSDB file might have multiple dump off ranges. This API returns the number of dump off ranges and their values.

Arguments

`count`

The number of dump off ranges in the file.

`*range`

An array of dump off ranges in the file. The number of elements in the array is the same as the argument `count`. An **fsdbDumpOffRange** structure consists of a begin time and an end time of a dump off range. **fsdbDumpOffRange** is defined in **fsdbShr.h**.

Return Value

FSDB_RC_SUCCESS if successful, **FSDB_RC_FAILURE** otherwise.

See Also

`ffw_EnableDumpOff`

`ffw_DisableDumpOff`

ffrGetEvcdPortStrength

Interface

```
static fsdbRC ffrObject::ffrGetEvcdPortStrength (byte_T *evcd_bit_val, fsdbBitType  
*ret_port_val, fsdbStrengthType *ret_s0, fsdbStrengthType *ret_s1);
```

Purpose

A value change in an extended VCD file consists of a state (value), a strength that indicates the strength0 specification for its port (signal), and a strength that indicates the strength1 specification.

If the FSDB file is an extended VCD one, the value changes described above are encoded in a special way. To retrieve such value changes, you must call **ffrVCIterOne::ffrGetVC()** to get the value change of a signal and interpret the value change with this API to get the value of the state and two strengths. You could call **ffrObject::ffrIsEvcd()** to see if the FSDB file is an extended VCD one.

For the details about extended VCD, refer to IEEE-1364 "Value change dump (VCD) files" section.

Arguments

`*evcd_bit_val`

The value change of the signal. It is usually returned by **ffrVCIterOne::ffrGetVC()** or similar APIs.

`*ret_port_val`

The state of the port. There are 25 possible states. They are defined in **fsdbShr.h** and ranging from **FSDB_BT_EVCD_L** to **FSDB_BT_EVCD_f**.

`*ret_s0`

The strength0 specification for the port. **fsdbStrengthType** is defined in **fsdbShr.h**.

`*ret_s1`

The strength1 specification for the port. **fsdbStrengthType** is defined in **fsdbShr.h**.

Return Value

The API always returns success. Check if the FSDB file is an extended VCD one before using this API to avoid unexpected results.

Example

```
byte_T *vc;  
fsdbBitType p;  
fsdbStrengthType s0, s1;  
  
vc_trvs_hdl->ffrGetVC(&vc)  
ffrObject::ffrGetEvcdPortStrength(vc, &p, &s0, &s1);
```

See Also

ffwCreateEvcdVarValByVarIdcode

ffwCreateEvcdVarValByVarHdl

ffrGetFileDumperType

Interface

`str_T ffrObject::ffrGetFileDumperType() const;`

Purpose

Get the dumper type.

Return Value

Non-null if successful; null otherwise.

ffrGetFileSimType

Interface

```
str_T ffrObject::ffrGetFileSimType() const;
```

Purpose

Get the simulator type.

Return Value

Non-null if successful; null otherwise.

ffrGetFileType

Interface

```
fsdbFileType ffrObject::ffrGetFileType();
```

Purpose

Get the file type. The file type is defined as **fsdbFileType** in the **fsdbShr.h** header file. This API is used from FSDB version 2.0.

Return Value

File type of the FSDB file.

See Also

ffw_Open, ffw_SetDesignType

ffrGetFSDBInfo

Interface

```
static fsdbRC ffrObject::ffrGetFSDBInfo (str_T fsdb_fname, ffrFSDBInfo &fsdb_info);
```

Purpose

Get the FSDB information before opening the file if it is of FSDB type. The FSDB information includes file type, min and max xtag, FSDB version, simulator version, simulation date, scale unit, and so on. This API is used from FSDB version 2.0.

Arguments

`fsdb_fname`

File name of the FSDB file.

`fsdb_info`

Information about the FSDB file. **ffrFSDBInfo** is defined in the **ffrAPI.h** header file.

Return Value

FSDB_RC_SUCCESS if the file is of FSDB type; **FSDB_RC_FAILURE** if the file is not of FSDB type.

ffrGetIdent

Interface

```
static str_T ffrObject::ffrGetIdent();
```

Purpose

Get the identification string defined by Novas. The identification string contains information about FSDB version and code compilation date, etc.

Return Value

Non-null if successful; null otherwise.

See Also

`ffw_GetIdent`

ffrGetInterface

Interface

```
static void* ffrGetInterface(ffrObject* obj, ffrObjectInterfaceID InterfaceID);
```

Purpose

Provide interfaces to access functions in ffrObject. The functions in new interfaces are identical to the ones available in ffrObject. This API and interfaces are for future updating and maintenance, and it does not affect the existing usage.

After opening the FSDB file, the interface pointer is returned by passing the ffrObject and relevant InterfaceID to this API. The type of InterfaceID is defined in the *ffrKit.h* header file. If the specified InterfaceID does not exist, a warning message appears.

Arguments

`*obj`

A pointer returned by ffrOpen3().

`InterfaceID`

The enum for specific interface version. ffrObjectInterfaceID is defined in *ffrKit*. header file.

Return Value

Non-null if successful; null otherwise.

ffrGetLoopMarkerArray

Interface

```
fsdbRC ffrObject::ffrGetLoopMarkerArray (uint_T &count, fsdbVarIdcode *&idcode_array);
```

Purpose

Use this API to get the idcode array of the loop marker variables stored in the FSDB file. You can then use the idcode array to load their value changes.

Arguments

`count`

The number of loop marker variables in the FSDB file. 0 if the FSDB file does not have any.

`*idcode_array`

The array of idcode of the loop marker variables. The returned address is a temporary storage. You must copy its content to your own memory if you need to use it in future.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrObject::ffrHasLoopMarkerVar

ffw_CreateLoopMarkerVarValueByIdcode

ffw_CreateLoopMarkerVarValueByHandle

ffrGetMaxFsdbTag64

Interface

```
fsdbRC ffrObject::ffrGetMaxFsdbTag64 (fsdbTag64 *tag64);
```

Purpose

Get the maximum simulation time of the FSDB file in **fsdbTag64** format. **fsdbTag64** is defined in the **fsdbShr.h** header file. The API usually applies to FSDB files of digital simulation results like Verilog/VHDL.

Arguments

*tag64

The maximum simulation time of the FSDB file in **fsdbTag64** format.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffw_CreateXCoorByHnL

ffrObject::ffrGetMinFsdbTag64

ffrGetMaxFsdbTagDouble

Interface

```
fsdbRC ffrObject::ffrGetMaxFsdbTagDouble (fsdbTagDouble *tag_double);
```

Purpose

Get the maximum simulation time of the FSDB file in **fsdbTagDouble** format. **fsdbTagDouble** is defined in the **fsdbShr.h** header file. The API usually applies to FSDB files of analog simulation results like Nanosim/SPICE.

Arguments

`*tag_double`

The maximum simulation time of the FSDB file in **fsdbTagDouble** format.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

ffrGetMaxFsdbTagFloat

Interface

```
fsdbRC ffrObject::ffrGetMaxFsdbTagFloat (fsdbTagFloat *tag_float);
```

Purpose

Get the maximum simulation time of the FSDB file in **fsdbTagFloat** format. **fsdbTagFloat** is defined in the **fsdbShr.h** header file. The API usually applies to FSDB files of analog simulation results like Nanosim/SPICE.

Arguments

`*tag_float`

The maximum simulation time of the FSDB file in **fsdbTagFloat** format.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

ffrGetMaxVarIdcode

Interface

```
fsdbVarIdcode ffrObject::ffrGetMaxVarIdcode() const;
```

Purpose

Get the maximum var idcode of the FSDB file. Each unique signal in FSDB has a unique idcode, therefore, the maximum var idcode is equal to the unique signal count. Note that var idcode begins from **FSDB_MIN_VAR_IDCODE**. **FSDB_MIN_VAR_IDCODE** is defined in **fsdbShr.h** and its value is usually 1.

Return Value

The maximum var idcode of the FSDB file if successful.

ffrGetMinFsdbTag64

Interface

```
fsdbRC ffrObject::ffrGetMinFsdbTag64 (fsdbTag64 *tag64);
```

Purpose

Get the minimum simulation time of the FSDB file in **fsdbTag64** format. **fsdbTag64** is defined in the **fsdbShr.h** header file. The API usually applies to FSDB files of digital simulation results like Verilog/VHDL.

Arguments

**tag64*

The minimum simulation time of the FSDB file in **fsdbTag64** format.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

`ffw_CreateXCoorByHnL`

`ffrObject::ffrGetMaxFsdbTag64`

ffrGetMinFsdbTagDouble

Interface

```
fsdbRC ffrObject::ffrGetMinFsdbTagDouble (fsdbTagDouble *tag_double);
```

Purpose

Get the minimum simulation time of the FSDB file in **fsdbTagDouble** format. **fsdbTagDouble** is defined in the **fsdbShr.h** header file. The API usually applies to FSDB files of analog simulation results like Nanosim/SPICE.

Arguments

**tag_double*

The minimum simulation time of the FSDB file in **fsdbTagDouble** format.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

ffrGetMinFsdbTagFloat

Interface

```
fsdbRC ffrObject::ffrGetMinFsdbTagFloat (fsdbTagFloat *tag_float);
```

Purpose

Get the minimum simulation time of the FSDB file in **fsdbTagFloat** format. **fsdbTagFloat** is defined in the **fsdbShr.h** header file. The API usually applies to FSDB files of analog simulation results like Nanosim/SPICE.

Arguments

`*tag_float`

The minimum simulation time of the FSDB file in `fsdbTagFloat` format.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

ffrGetOEMActiveObj

Interface

```
static ffrObject* ffrObject::ffrGetOEMActiveObj();
```

Purpose

Return the OEM active object set by **ffrSetOEMActiveObj**.

Return Value

The OEM active object set by **ffrObject::ffrSetOEMActiveObj**. If not set, NULL is returned.

See Also

`ffrObject::ffrSetOEMActiveObj`

ffrGetOEMFuncTbl

Interface

```
static ffrOEMFuncTbl* ffrObject::ffrGetOEMFuncTbl();
```

Purpose

Return the OEM function table kept by the FSDB Reader. See **ffrOEMInit** for details.

Return Value

The OEM function table kept by the FSDB Reader. Its entries are NULL if the table is not filled by OEM.

ffrGetPropVarStatInfo

Interface

```
fsdbRC ffrObject::ffrGetPropVarStatInfo (fsdbVarIdcode var_idcode, ffrPropVarStatInfo  
&ret_info);
```

Purpose

Get the statistic information of a property var whose idcode is var_idcode.

Arguments

var_idcode

The idcode of the signal whose information is to be retrieved. The information is stored in **ret_info**. **ffrVarInfo**, which is defined in **ffrAPI.h**.

ret_info

The retrieved property statistic information to be filled by the FSDB Reader.

Return Value

FSDB_RC_PROBE_FOUND if successful; **FSDB_RC_PROBE_NOT_FOUND** if otherwise.

ffrGetRelationName

Interface

```
fsdbRC ffrObject::ffrGetRelationName (fsdbRelationHdl relation_hdl, str_T &name);
```

Purpose

Return the name of the relation specified by *relation_hdl*.

Arguments

`relation_hdl`

Handle of the relation.

`name`

Transaction name of the relation the FSDB Reader returns.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_RELATION_NOT_FOUND** if `relation_hdl` is invalid.

See Also

`ffw_CreateRelation`

ffrGetScaleUnit

Interface

```
str_T ffrObject::ffrGetScaleUnit() const;
```

Purpose

Get the scale unit of the FSDB file. The value can be 10ns, 1ps and so on.

Return Value

A pointer to the string which represents the scale unit. It returns non-null if successful, null otherwise.

See Also

`ffw_SetScaleUnit`

ffrGetScopeSeparator

Interface

```
str_T ffrObject::ffrGetScopeSeparator() const;
```

Purpose

Get the scope separator stored of the FSDB file. Scope separator is stored by simulator during dumping.

Return Value

Non-null if successful; null otherwise.

See Also

ffw_SetScopeSeparator

ffrGetSessionListInfo

Interface

```
ffrSessionInfo* ffrObject::ffrGetSessionListInfo() const;
```

Purpose

The FSDB Writer writes out value changes when buffer is full and this is called a flush session. This API could return the start_xtag/close_xtag of each flush session in the FSDB file.

The information is useful when the application needs to call **ffrSetViewWindow** to limit the amount of value changes loaded. It is recommended that the application specifies the start_xtag of view window as the start_xtag of a certain flush session and close_xtag of view window as the close_xtag of a certain flush session. The reason is that when loading value changes, the FSDB Reader always loads a complete flush session. Therefore, the start_xtag/close_xtag of flush sessions could be used as a hint of the width of a view window.

Return Value

The information of all flush sessions in the FSDB file. **ffrSessionInfo** is defined in **ffrAPI.h**. Null is returned on failure.

See Also

ffrObject::ffrSetViewWindow

ffw_Sync

ffrGetSimDate

Interface

```
str_T ffrObject::ffrGetSimDate() const;
```

Purpose

Get the simulation date. Simulation date is stored by simulator during dumping.

Return Value

Non-null if successful; null otherwise.

See Also

ffw_SetSimulationDate

ffrGetSimVersion

Interface

```
str_T ffrObject::ffrGetSimVersion() const;
```

Purpose

Get the simulator version. Simulator version is stored by simulator during dumping.

Return Value

Non-null if successful; null otherwise.

See Also

ffw_SetSimulatorVersion

ffrGetSupportInfo

Interface

```
fsdbRC ffrObject::ffrGetSupportInfo (fsdbSupInfo &support_info);
```

Purpose

Retrieve the support information in the FSDB file. You must call the **ffrHasSupportInfo** function to check if the file has support information or not before using this function. If not, unexpected results can occur. This API is used from FSDB version 2.0.

Arguments

support_info

The support information in the FSDB file that is to be filled by the FSDB Reader. The size of **fsdbSupInfo** structure is **FSDB_SUPINFO_LEN** bytes and it contains the following fields:

Field	Data Type	Length	Note
architecture	char*	FSDB_SUPINFO_ARCHITECTURE_LEN	Architecture of the host on which this FSDB file is created.
creation_time	char*	FSDB_SUPINFO_CREATION_TIME_LEN	Creation time of this FSDB file.
env_var	char*	FSDB_SUPINFO_ENV_VAR_LEN	A list of FSDB related environment variables of the environment in which this FSDB file is created.
glitch_num	ushort_T	FSDB_SUPINFO_GLITCH_NUM_LEN	The glitch number with which this FSDB file is created.
host_name	char*	FSDB_SUPINFO_HOST_NAME_LEN	Name of the host on which this FSDB file is created.
ident	char*	FSDB_SUPINFO_IDENT_LEN	Information about the FSDB Writer internally used by Novas.
os_type	char*	FSDB_SUPINFO_OS_TYPE_LEN	Operating system type of the host on which this FSDB file is created.
os_version	char*	FSDB_SUPINFO_OS_VERSION_LEN	Operating system version of the host on which this FSDB file is created.
producer	char*	FSDB_SUPINFO_PRODUCER_LEN	Path of the program that created the FSDB file. Set by <code>ffw_SetProducer()</code> .
user_defined	char*	FSDB_SUPINFO_USER_DEFINED_LEN	Information

From `creation_time` to `ident`, they are written by the FSDB Writer by default. The `producer` is stored by calling **ffw_SetProducer** in the FSDB Writer. `user_defined` is stored by calling **ffw_AddSupportInfo** in the FSDB Writer.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrObject::ffrHasSupportInfo

ffw_AddSupportInfo ffw_SetProducer

ffrGetTransInfo

Interface

```
fsdbRC ffrObject::ffrGetTransInfo (fsdbTransId trans_id, ffrTransInfo *&info);
```

Purpose

Gets the values of the transaction specified by *trans_id*. The result is filled in *info* by the FSDB Reader. See **ffrVCIterOne::ffrGetTransInfo** for details. This API is used from FSDB version 2.0.

Arguments

trans_id

The id of the transaction.

**info*

An **ffrTransInfo** that holds the result returned by the FSDB Reader. See **ffrTransInfo** in **Transaction Traversal** section for details.

Return Value

FSDB_RC_SUCCESS on success; **FSDB_RC_FAILURE** otherwise.

See Also

ffrVCIterOne::ffrGetTransInfo

ffrGetTreeCBFunc

Interface

```
fsdbRC ffrObject::ffrGetTreeCBFunc (fsdbTreeCBFunc *ret_tree_cb_func, void  
**ret_tree_client_data);
```

Purpose

Returns the tree callback function and tree callback data of the FSDB Reader object. Tree callback function and tree callback data is useful when traversing the design hierarchy in the FSDB file.

Tree callback function and tree callback data of an FSDB Reader object is either set upon opening an FSDB file (with **ffrObject::ffrOpen** or **ffrObject::ffrOpen2**) or set explicitly by the application (with **ffrObject::ffrSetTreeCBFunc**).

Arguments

`*ret_tree_cb_func`

Tree callback function of the FSDB Reader object.

`**ret_tree_client_data`

The tree client data of the FSDB Reader object.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

`ffrObject::ffrOpen`

`ffrObject::ffrOpen2`

`ffrObject::ffrSetTreeCBFunc`

`ffrObject::ffrReadScopeTree2`

`ffrObject::ffrReadScopeVarTree2`

ffrGetTriggerPointVarIdcode

Interface

`fsdbRC ffrObject::ffrGetTriggerPointVarIdcode (fsdbVarIdcode &idcode);`

Purpose

Get the idcode of the trigger point signal in the FSDB file, if any. Trigger point signal is supported in Novas.qds only.

Arguments

`idcode`

If the FSDB file contains a trigger point signal, the value of *idcode* is the idcode of the signal. If not, the value of idcode is **FSDB_INVALID_VAR_IDCODE**.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrObject::ffrHasTriggerPointVar

ffrGetVarInfoByVarIdcode

Interface

```
fsdbRC ffrObject::ffrGetVarInfoByVarIdcode (fsdbVarIdcode var_idcode, ffrVarInfo  
**ret_var_info);
```

Purpose

Retrieve the information of the signal specified by var_idcode. The information is stored in ret_var_info. **ffrVarInfo** is defined in **ffrAPI.h**.

ret_var_info is only valid when the information of the signal is already loaded by **ffrObject::ffrReadScopeVarTree** or **ffrObject::ffrReadVarByLogUOff**; otherwise, its value might be wrong.

Arguments

var_idcode

The idcode of the signal whose information is to be retrieved.

**ret_var_info

The retrieved information that is to be filled by the FSDB Reader. The address of ret_var_info should not be kept for future reference. If it is needed later, caller must copy the content to its local memory instead of keeping the address of the pointer.

Return Value

FSDB_RC_SUCCESS if successful. **FSDB_RC_FAILURE** if var_idcode is invalid.

See Also

ffrObject::ffrReadScopeVarTree

ffrObject::ffrReadVarByLogUOff

ffrGetVCCount

Interface

```
uint_T ffrObject::ffrGetVCCount (fsdbVarIdcode var_idcode);
```

Purpose

Return the number of the loaded value changes of the signal specified by *var_idcode*. Only the value changes that are loaded into memory are counted. Whether **ffrObject::ffrSetViewWindow** is called or not also affects the result of this API. This API is used from FSDB version 2.0.

Arguments

var_idcode

The idcode of the signal.

Return Value

The number of the loaded value changes of the signal. It returns 0 if:

1. The signal does not have any value change.
2. *var_idcode* is invalid.
3. The FSDB version is earlier than 2.0.

See Also

`ffrObject::ffrSetViewWindow`

`ffrObject::ffrLoadSignals`

ffrGetVersion

Interface

```
ffrObject::ffrGetVersion();
```

Purpose

Get the FSDB library version.

Return Value

Non-null if successful; null otherwise.

ffrGetVrtlFileType

Interface

```
ffrObject::ffrGetVrtlFileType();
```

Purpose

Get the virtual file type.

Return Value

The virtual file type is defined as **fsdbVrtlFileType** in the **fsdbShr.h** header file.

ffrGetVwlsSet

Interface

```
bool_T ffrObject::ffrGetVwlsSet ();
```

Purpose

Check to see if of the value change has set a view window or not. A view window can be set with **ffrObject::ffrSetViewWindow**. This API is used from FSDB version 2.0.

Return Value

TRUE if view window is set; FALSE otherwise or FSDB version is earlier than 2.0.

See Also

```
ffrObject::ffrSetViewWindow
```

ffrGetXTagType

Interface

```
fsdbXTagType ffrObject::ffrGetXTagType(void) const;
```

Purpose

Get the xtag type of the FSDB file. The x coordinate can be 32-bit or 64-bit integer, float or double, so in general, it is called an xtag. The time type is defined in the **fsdbShr.h header** file: **FSDB_XTAG_TYPE_L**, **FSDB_XTAG_TYPE_HL**, **FSDB_XTAG_TYPE_FLOAT** and **FSDB_XTAG_TYPE_DOUBLE**.

This API is used from FSDB version 2.0.

Return Value

fsdbXTagType of the FSDB file.

ffrHasBeenComputed

Interface

```
bool_T ffrObject::ffrHasBeenComputed (fsdbVarIdcode var_idcode);
```

Purpose

Checks if the signal specified by *var_idcode* is computed and already has value changes inserted by OEM Computed Engine.

This is for some OEM Computed Engine use only and most applications do not have to use this API. Computed signals are supported from FSDB version 2.0 and only work in some OEM versions of Novas.

This API only detects if a signal has value changes or not regardless if the view window remains the same or not. The application must handle this situation.

Arguments

var_idcode

The idcode of the signal to be tested.

Return Value

TRUE if the signal is a computed one and already has value changes inserted by OEM Computed Engine.

FALSE if:

1. The computed signal does not any value change.
2. The signal is not a computed one.
3. *var_idcode* is invalid.
4. The FSDB version is earlier than 2.0.

See Also

ffrOEMInit ffrObject::ffrInsertComputedVC

ffrObject::ffrSetViewWindow

ffrHasDataTypeDef

Interface

```
bool_T ffrObject::ffrHasDataTypeDef();
```

Purpose

Check if the FSDB file contains data type definition or not.

Return Value

TRUE if the FSDB file contains data type definition; FALSE otherwise.

See Also

ffrObject::ffrReadDataTypeDefByBlkIdx

ffw_GetDataTypeCreationReady

ffrHasDumpOffRange

Interface

```
bool_T ffrObject::ffrHasDumpOffRange() const;
```

Purpose

Check if the file has any dump off range. The FSDB Writer APIs **ffw_EnableDumpOff** and **ffw_DisableDumpOff** are used to create dump off ranges. Dump off is added from FSDB version 2.0.

Return Value

TRUE if the FSDB file has dump off ranges; FALSE otherwise.

See Also

ffrObject::ffrGetDumpOffRange

ffrHasGlitch

Interface

```
bool_T ffrObject::ffrHasGlitch (fsdbVarIdcode idcode);
```

Purpose

Checks if the loaded value changes of the signal specified by `idcode` contains any glitch. Note that it is the "loaded value changes" of the signal instead of "value changes in the whole file" that are checked. Whether **`ffrObject::ffrSetViewWindow`** is called or not affects the result of this API.

Arguments

`idcode`

The `idcode` of the signal to be tested.

Return Value

TRUE if the loaded value changes of the signal contain glitches or the FSDB file version is earlier than 2.0; FALSE otherwise.

See Also

`ffrVCIterOne::ffrGetGlitchNum` `ffw_SetGlitchNum`

ffrHasLoopMarkerVar

Interface

```
bool_T ffrObject::ffrHasLoopMarkerVar();
```

Purpose

Detect if the FSDB file contains any loop marker variables or not. Loop marker variable is added from FSDB version 2.0.

Return Value

TRUE if the file has loop marker variables; FALSE otherwise.

See Also

`ffrObject::ffrGetLoopMarkerArray`

`ffw_CreateLoopMarkerVarValueByIdcode`

`ffw_CreateLoopMarkerVarValueByHandle`

ffrHasNewTree

Interface

```
bool_T ffrObject::ffrHasNewTree();
```

Purpose

Check if new trees exist in the file. It is used when the FSDB file is still in progress. This API is used from FSDB version 2.0.

Return Value

TRUE if there comes a new tree; FALSE otherwise.

ffrHasScopeTree

Interface

```
bool_T ffrObject::ffrHasScopeTree();
```

Purpose

Check the FSDB file to see if it contains scope tree or not. Scope tree is supported from FSDB version 2.0, which enables the application to traverse only scopes in the design hierarchy and is much faster than traversing both scopes and signals. It is useful if the application does not need signal information.

Traversing scope tree with **ffrObject::ffrReadScopeTree** only involves scope hierarchy. It is not like **ffrObject::ffrReadScopeVarTree** which traverses both scopes and signals. Therefore, the traversing speed is much faster.

Return Value

TRUE if the FSDB file contains scope tree; FALSE otherwise.

See Also

`ffrObject::ffrReadScopeTree`

`ffrObject::ffrReadScopeVarTree`

ffrHasSupportInfo

Interface

```
bool_T ffrObject::ffrHasSupportInfo();
```

Purpose

Check to see if this FSDB file contain any support information or not. The support information is added from FSDB version 2.0.

The FSDB Writer records support information by default even if neither **ffw_AddSupportInfo** nor **ffw_SetProducer** is called. This function only returns FALSE when the FSDB version is earlier than 2.0.

Return Value

TRUE if the FSDB file has support information; FALSE otherwise.

See Also

ffrObject::ffrGetSupportInfo

ffw_AddSupportInfo ffw_SetProducer

ffrHasTriggerPointVar

Interface

```
bool_T ffrObject::ffrHasTriggerPointVar();
```

Purpose

Check the FSDB file to see if it has trigger point signal. Trigger point signal is supported in Novas.qds only.

Return Value

TRUE if the FSDB file has trigger point signal; FALSE otherwise.

See Also

ffrObject::ffrGetTriggerPointVarIdcode

ffrInfoSuppress

Interface

```
static void ffrObject::ffrInfoSuppress (bool_T if_suppress);
```

Purpose

Turn on/off information issued by the FSDB Reader. This API is used from FSDB version 2.3.

Arguments

`if_suppress`

TRUE to turn off information; FALSE to turn on.

See Also

`ffrObject::ffrWarnSuppress`

ffrInitialize

Interface

```
static void ffrObject::ffrInitialize();
```

Purpose

Initialize necessary data structures for the FSDB Reader so that the FSDB file can be read smoothly.

ffrInsertComputedPropVC

Interface

```
fsdbRC ffrObject::ffrInsertComputedPropVC (fsdbVarIdcode var_idcode, fsdbXTag  
*begin_time, fsdbXTag *end_time, str_T result_str);
```

Purpose

Insert a computed property value change of the signal specified by *var_idcode* into the FSDB Reader memory, so it can be displayed in a waveform window. See **ffrObject::ffrInsertComputedVC** for details.

This API is used for some OEM Computed Engine only and most applications do not have to use this API. Computed signals are supported from FSDB version 2.0 and only works in some OEM versions of Novas.

Arguments

`var_idcode`

Idcode of the computed signal whose value change is to be inserted.

`*begin_time`

Begin time of the property value change to be inserted.

`*end_Time`

End time of the property value change to be inserted.

`result_str`

Result string of the property value change to be inserted.

Return Value

FSDB_RC_SUCCESS on success. **FSDB_RC_MEMORY_IS_EXHAUSTED** if memory is exhausted. **FSDB_RC_FAILURE** if *var_idcode* is invalid or not found or if the signal specified by *var_idcode* is not a property signal.

See Also

`ffrObject::ffrInsertComputedVC`

ffrInsertComputedTransVC

Interface

`fsdbRC ffrObject::ffrInsertComputedTransVC (fsdbVarIdcode var_idcode, fsdbXTag *begin_time, fsdbXTag *end_time, str_T label, fsdbAttrHdlVal attr_arr[], ushort_T attr_count, fsdbTransStatus status);`

Purpose

Insert a computed transaction value change of the signal specified by *var_idcode* into the FSDB Reader memory so that it could be displayed in a waveform window. See **ffrObject::ffrInsertComputedVC** for details.

This API is used for some OEM Computed Engine only and most applications do not have to use this API. Computed signals are supported from FSDB version 2.0 and only works in some OEM versions of Novas.

Arguments

`var_idcode`

Idcode of the computed signal whose value change is to be inserted.

`*begin_time, *end_Time, label, attr_attr, attr_count, status`

See **ffrTransInfo** in **Transaction Traversal** section for details.

Return Value

FSDB_RC_SUCCESS on success. **FSDB_RC_MEMORY_IS_EXHAUSTED** if memory is exhausted. **FSDB_RC_FAILURE** if *var_idcode* is invalid or not found or if the signal specified by *var_idcode* is not a stream signal.

See Also

ffrObject::ffrInsertComputedVC

ffrInsertComputedVC

Interface

```
fsdbRC ffrObject::ffrInsertComputedVC (fsdbVarIdcode var_idcode, fsdbXTag *time,  
byte_T *vc);
```

Purpose

Insert a computed value change of the signal specified by *var_idcode* into the FSDB Reader memory so that it could be displayed in a waveform window. The value change is computed by OEM Computed Engine. See **ffrOEMInit** for the detail of OEM computed value change scheme.

This is for some OEM Computed Engine use only and most applications do not have to use this API. Computed signals are supported from FSDB version 2.0 and only work in some OEM versions of Novas.

Arguments

var_idcode

Idcode of the computed signal whose value change is to be inserted.

**time*

Time step of the value change to be inserted.

**vc*

Value change to be inserted. It must match the data type of the computed signal. For example, if the data type of the signal is **FSDB_DT_VERILOG_STANDARD**, valid values are **FSDB_BT_VCD_0** to **FSDB_BT_VCD_Z**. If the data type is **FSDB_DT_VHDL_STD_LOGIC**, valid values are **FSDB_BT_VHDL_STD_LOGIC_U** to **FSDB_BT_VHDL_STD_LOGIC_DASH**.

Return Value

FSDB_RC_SUCCESS on success. **FSDB_RC_MEMORY_IS_EXHAUSTED** if memory is exhausted. **FSDB_RC_FAILURE** if `var_idcode` is invalid or not found.

See Also

`ffrOEMInit ffrObject::ffrHasBeenComputed`

ffrlsComputedVar

Interface

```
bool_T ffrObject::ffrlsComputedVar (fsdbVarIdcode var_idcode);
```

Purpose

See if the signal whose `idcode` is `var_idcode` is a computed signal or not.

This is for some OEM Computed Engine use only and most applications do not have to use this API. Computed signals are supported from FSDB version 2.0 and only work in some OEM versions of Novas.

Arguments

`var_idcode`

Idcode of the signal to be tested.

Return Value

TRUE if the signal is a computed one.

FALSE if:

1. The signal is not a computed one.
2. `var_idcode` is invalid.
3. The FSDB version is earlier than 2.0.

See Also

`ffrObject::ffrlsMiddleVar`

`ffrObject::ffrlsMiddleOrComputedVar`

ffrlsEvcd

Interface

```
bool_T ffrObject::ffrlsEvcd();
```

Purpose

Check the FSDB file to see if it is an extended VCD one or not. It is an extended VCD one as long its value changes are created with **ffwCreateEvcdVarValByVarIdcode** or **ffwCreateEvcdVarValByVarHdl**. This API is used from FSDB version 2.0.

For the details about extended VCD, please refer to IEEE-1364 "*Value change dump (VCD) files*" section.

Return Value

TRUE if the FSDB file is an extended VCD one; FALSE otherwise.

See Also

ffrObject::ffrGetEvcdPortStrength

ffwCreateEvcdVarValByVarIdcode

ffwCreateEvcdVarValByVarHdl.

ffrlsFSDB

Interface

```
static bool_T ffrObject::ffrlsFSDB (str_T file_name);
```

Purpose

Check the file if it is an FSDB file.

Arguments

file_name

A pointer to a string which represents the FSDB file name.

Return Value

TRUE if the file is an FSDB file. FALSE if the file is not an FSDB file or the FSDB Reader fails to open the file.

See Also

ffrObject::ffrCheckFSDB

ffrObject::ffrCheckFile

ffrIsInSignalList

Interface

```
bool_T ffrObject::ffrIsInSignalList (fsdbVarIdcode idcode);
```

Purpose

Check the signal if it is in signal list. The signal is specified by its *idcode*. A signal is in signal list if it is added by **ffrObject::ffrAddToSignalList**.

Arguments

idcode

Var *idcode* which identifies a signal.

Return Value

TRUE if the signal is in signal list; FALSE otherwise.

See Also

ffrObject::ffrAddToSignalList

ffrIsMiddleOrComputedVar

Interface

```
bool_T ffrObject::ffrIsMiddleOrComputedVar (fsdbVarIdcode var_idcode);
```

Purpose

See if the signal whose *idcode* is *var_idcode* is a middle or computed signal or not.

This is for some OEM Computed Engine use only and most applications do not have to use this API. Computed and middle signals are supported from FSDB version 2.0 and only work in some OEM versions of Novas.

Arguments

var_idcode

Idcode of the signal to be tested.

Return Value

TRUE if the signal is a middle or computed one.

FALSE if:

1. The signal is neither a middle nor a computed one.
2. *var_idcode* is invalid.
3. The FSDB version is earlier than 2.0.

See Also

ffrObject::ffrlsComputedVar

ffrObject::ffrlsMiddleOrComputedVar

ffrlsMiddleVar

Interface

```
bool_T ffrObject::ffrlsMiddleVar (fsdbVarIdcode var_idcode);
```

Purpose

See if the signal whose idcode is *var_idcode* is a middle signal or not.

This is for some OEM Computed Engine use only and most applications do not have to use this API. Middle signals are supported from FSDB version 2.0 and only work in some OEM versions of Novas.

Arguments

var_idcode

Idcode of the signal to be tested.

Return Value

TRUE if the signal is a middle one. FALSE if:

1. The signal is not a middle one.
2. *var_idcode* is invalid.
3. The FSDB version is earlier than 2.0.

See Also

ffrObject::ffrlsComputedVar

ffrObject::ffrlsMiddleOrComputedVar

ffrlsThereSeqNum

Interface

```
bool_T ffrObject::ffrlsThereSeqNum();
```

Purpose

Check the file if it contains sequence number dumping.

Return Value

TRUE if the file contains sequence number dumping; FALSE otherwise.

See Also

ffw_DumpSeqNumTurnOn

ffrlsViewWindowSet

Interface

```
bool_T ffrObject::ffrlsViewWindowSet() const;
```

Purpose

Check if the application has set a view window or not. A view window can be set with **ffrObject::ffrSetViewWindow**. This API is used from FSDB version 2.0.

Return Value

TRUE if view window is set; FALSE otherwise.

See Also

ffrObject::ffrSetViewWindow

ffrIsVrtlFile

Interface

```
ffrObject::ffrIsVrtlFile(str_T fd_name);
```

Purpose

Check if the specified FSDB file is a virtual file or not.

Arguments

`fd_name`

The FSDB file name.

Return Value

TRUE if the file is a virtual file; FALSE otherwise.

ffrLoadSignals

Interface

```
fsdbRC ffrObject::ffrLoadSignals();
```

Purpose

Load the value changes of the signals that are in signal list onto memory. That means if a signal is not selected into signal list, then **ffrObject::ffrLoadSignals** does not load its value changes onto memory. A signal is in signal list if it is added by **ffrObject::ffrAddToSignalList**.

Return Value

FSDB_RC_SUCCESS if successful. **FSDB_RC_MEMORY_IS_EXHAUSTED** if memory is exhausted. **FSDB_RC_FAILURE** if other error occurs.

See Also

`ffrObject::ffrAddToSignalList`

`ffrObject::ffrIsInSignalList`

ffrNoChangeEpicVarName

Interface

```
fsdbRC ffrNoChangeEpicVarName();
```

Purpose

When the NanoSim FSDB (type FSDB_FT_NANOSIM) signal name includes a slash (/) character, the FSDB Reader changes it to a caret (^) character.

Calling this API prevents the FSDB Reader from converting this special character. This API is useful when users want to get the real NanoSim signal name.

Return Value

FSDB_RC_SUCCESS if successful. **FSDB_RC_FAILURE** if other error occurs.

ffrOpen

Interface

```
static ffrObject* ffrObject::ffrOpen (str_T fname, fsdbTreeCBFunc tree_cb_func, void  
*tree_client_data, ffrOpenOption opt = FFR_OPEN_OPT_DFT);
```

Purpose

Opens an FSDB file, creates an FSDB Reader object, and returns the reader object to caller. **ffrOpen** allows the application to register a tree callback function and a tree client data so that the application can traverse the design hierarchy.

During the open session, the FSDB Reader reads the design hierarchy. If the second argument *tree_cb_func* is not null, the FSDB Reader notifies the application what it reads from the design hierarchy by calling *tree_cb_func*. In *tree_cb_func*, the application can notify what the FSDB Reader gets (scope, signal, upscope, etc) using callback data type and then performs a type cast on the callback data based on the callback data type. All the design hierarchies will be read during the open session.

If the application needs to pass some user-defined data into *tree_cb_func*, *tree_client_data* is a good place. It could also be used as a communication buffer between different calls to *tree_cb_func*.

It is recommended to use **ffrOpen3** to replace this API.

Arguments

fname

A pointer to a string which represents the FSDB file name.

`tree_cb_func`

Tree callback function given by the application. **fsdbTreeCBFunc** is defined in the **fsdbShr.h** header file.

`*tree_client_data`

Tree client data that will be passed back in tree callback function.

`opt`

An open option used by Novas only. It has a default value defined and the application can just ignore it.

Return Value

Non-null if successful; null otherwise.

See Also

`ffrObject::ffrOpen2`

`ffrObject::ffrOpen3`

ffrOpen2

Interface

```
static ffrObject* ffrObject::ffrOpen2 (str_T fname, fsdbTreeCBFunc tree_cb_func, void
*tree_client_data, ffrOpenOption opt = FFR_OPEN_OPT_DFT, ffrCmprsFileArg *file_arg =
NULL, bool_T isSharedObj = 0);
```

Purpose

Open an FSDB file. See **ffrObject::ffrOpen** for details.

ffrObject::ffrOpen2 is almost equivalent to **ffrObject::ffrOpen**. The difference is that *tree_cb_func* is not called during the open session to save time opening an FSDB file. If an FSDB file is opened by **ffrObject::ffrOpen2**, the application has to call **ffrObject::ffrReadScopeTree** or **ffrObject::ffrReadScopeVarTree** to activate *tree_cb_func*. This API is used from FSDB version 2.0.

It is recommended to use **ffrOpen3** to replace this API.

Arguments

`fname`

File name of the FSDB file.

tree_cb_func

Tree callback function given by the application. **fsdbTreeCBFunc** is defined in the **fsdbShr.h** header file.

*tree_client_data

Tree client data that will be passed back in tree callback function.

opt

An open option used by Novas only. It has a default value defined and the application can just ignore it.

file_arg

File arguments. The following is an example of structure usages.

```
struct fFrCmprsFileArg {
    str_T file_name; // file name
    str_T decompress_file_loc; // location of the temporary decompressed FSDB
    file
    str_T decompressor_name; // if NULL, decompressor is gzip.
    str_T decompressor_option; // user specified decompressor options
    uint_T seconds_to_wait; // waiting time (in seconds) before killing the
    decompression process. 0 means to wait until decompression is finished.
    str_T ret_msg_buf; // user specified buffer for the decompressor result
    message.
    uint_T ret_msg_buf_size; // user specified buffer size. The message will
    be truncated if it is larger than the given size.
};
```

fFrCmprsFileArg

Open a compressed FSDB file in gzipped or bzip2 file format. This option is only used by the process that opens the gzipped or bzip2 FSDB file, and other processes should not open the generated temporary file. The decompressed FSDB file cannot be used by another application or user, because a hidden temporary file is generated in the specified directory.

This option is available only on LINUX and LINUX64 platforms.

isSharedObj

Specify whether the same address of the reader object is obtained if an FSDB file is opened multiple times in the same job. When **isSharedObj** is set to 1, this API works the same as **fFrOpenNonSharedObj**.

It is recommended to use **fFrObject::fFrOpenNonSharedObj** for multiple-thread programming on the FSDB Reader object.

Return Value

Non-null if successful; null otherwise.

See Also

ffrObject::ffrOpen

ffrObject::ffrOpen3

ffrOpen3

Interface

```
static ffrObject* ffrObject::ffrOpen3 (str_T fname);
```

Purpose

Open an FSDB file. See **ffrObject::ffrOpen2** for details.

ffrObject::ffrOpen3 is similar to **ffrObject::ffrOpen2**. They differ in that *tree_cb_func* and *tree_client_data* are **NULL**. Their values can be set later by calling **ffrObject::ffrSetTreeCBFunc**. For those applications which only perform basic operations and do not need to traverse the design hierarchy, this API is very convenient.

It is recommended to use this API to replace **ffrOpen** and **ffrOpen2**.

Arguments

fname

File name of the FSDB file.

Return Value

Non-null if successful; null otherwise.

See Also

ffrObject::ffrOpen

ffrObject::ffrOpen2

ffrOpenNonSharedObj

Interface

```
static ffrObject* ffrOpenNonSharedObj (str_T fname);
```

Purpose

Open an FSDB file. See **ffrObject::ffrOpen3** for details.

ffrObject::ffrOpenNonSharedObj is similar to **ffrObject::ffrOpen3**. The difference is that **ffrObject::ffrOpenNonSharedObj** returns a different address of the reader object if an FSDB file is opened multiple times in the same job.

It is recommended to use this API for multiple-thread programming on the FSDB Reader object.

Arguments

fname

File name of the FSDB file.

Return Value

Non-null if successful; null otherwise.

ffrOver2GIdcode

Interface

```
bool_T ffrOver2GIdcode() const;
```

Purpose

Checks whether the number of id code in FSDB is over 2G or not.

Return Value

True when the id code is over 2G, False when the id code is not over 2G.

ffrReadDataTypeDefByBlkIdx

Interface

```
fsdbRC ffrObject::ffrReadDataTypeDefByBlkIdx (uint_T &blk_idx);
```

Purpose

Read the data type definitions in the FSDB file. In FSDB, all data type definitions are stored in data type blocks. The argument *blk_idx* means data type block index. It is usually zero, which means to read from the first data type block to the last one. If it is not zero, say 1, it means to read from data type block 1 to the last one. At the end of this call, argument

blk_idx will be updated to indicate the biggest block index it has read so far, which is useful information when the FSDB file is still growing.

Arguments

blk_idx

When calling the API, the application needs to supply a value (usually 0) to indicate which data type block to begin to read data type definitions. After the API finishes, it contains the biggest index of data type block the FSDB Reader has read so far.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

When calling the API, application needs to supply a value (usually 0) to indicate which data type block to begin to read data type definitions. After the API finishes, it contains the biggest index of data type block the FSDB reader has read so far.

See Also

ffrObject::ffrReadDataTypeDefByBlkIdx2

ffrReadDataTypeDefByBlkIdx2

Interface

```
fsdbRC ffrObject::ffrReadDataTypeDefByBlkIdx2 (fsdbTreeCBFunc tree_cb_func, void  
*tree_client_data, uint_T &blk_idx);
```

Purpose

This function is similar to **ffrObject::ffrReadDataTypeDefByBlkIdx** with the difference that caller could ask the function to use *tree_cb_func* and *tree_client_data* instead of that of the FSDB reader object to handle data type definitions. This is useful when caller needs to handle data type definitions in its own way but does not want to change the tree function pointer and tree callback data of the FSDB reader object. The tree function pointer and tree callback data of the FSDB reader object remains intact after the function finishes.

Arguments

tree_cb_func

Tree callback function pointer that will be used in this function.

**tree_client_data*

Tree client data that will be used in this function.

`blk_idx`

When calling the API, application needs to supply a value (usually 0) to indicate which data type block to begin to read data type definitions. After the API finishes, it contains the biggest index of data type block the FSDB reader has read so far.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

`ffrObject::ffrReadDataTypeDefByBlkIdx`

ffrReadScopeTree

Interface

```
fsdbRC ffrObject::ffrReadScopeTree();
```

Purpose

Read scope information in the FSDB file. Unlike **ffrObject::ffrReadScopeVarTree**, it does not read the signals in each scope. Fsdb reader activates the tree callback function and pass scope information to application through the function.

This function is useful if application opens an FSDB file by **ffrObject::ffrOpen2** or **ffrObject::ffrOpen3**. These two functions do not traverse the design hierarchy upon opening the file. When it needs to retrieve the scope information later, it can call this function to do the job.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

`ffrObject::ffrOpen2`

`ffrObject::ffrReadScopeTree2`

`ffrObject::ffrReadScopeVarTree`

ffrReadScopeTree2

Interface

```
fsdbRC ffrObject::ffrReadScopeTree2 (fsdbTreeCBFunc tree_cb_func, void  
*tree_client_data);
```

Purpose

This function is similar to **ffrObject::ffrReadScopeTree** with the difference that caller could ask the function to use *tree_cb_func* and *tree_client_data* instead of that of the FSDB reader object to handle scope information. This is useful when caller needs to handle scope information in its own way but does not want to change the tree function pointer and tree callback data of the FSDB reader object. The tree function pointer and tree callback data of the FSDB reader object remains intact after the function finishes.

Arguments

tree_cb_func

Tree callback function pointer that is used in this function.

**tree_client_data*

Tree client data that is used in this function.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrObject::ffrReadScopeTree

ffrReadScopeVarTree

Interface

```
fsdbRC ffrObject::ffrReadScopeVarTree();
```

Purpose

Read both scope/signal information in the FSDB file. Unlike **ffrObject::ffrReadScopeTree**, it also reads the signals in each scope. FSDB reader will activate the tree callback function and pass scope/signal information to application through the function.

This function is useful if application opens an FSDB file by **ffrObject::ffrOpen2** or **ffrObject::ffrOpen3**. These two functions do not traverse the design hierarchy upon opening the file. When it needs to retrieve the scope/signal information later, it can call this function to do the job.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

`ffrObject::ffrOpen2`

`ffrObject::ffrReadScopeVarTree2`

`ffrObject::ffrReadScopeTree`

ffrReadScopeVarTree2

Interface

```
fsdbRC ffrObject::ffrReadScopeVarTree2 (fsdbTreeCBFunc tree_cb_func, void  
*tree_client_data);
```

Purpose

This function is similar to **ffrObject::ffrReadScopeVarTree** with the difference that caller could ask the function to use *tree_cb_func* and *tree_client_data* instead of that of the FSDB reader object to handle scope/signal information. This is useful when caller needs to handle scope/signal information in its own way but does not want to change the tree function pointer and tree callback data of the FSDB reader object. The tree function pointer and tree callback data of the FSDB reader object remains intact after the function finishes.

Arguments

`tree_cb_func`

Tree callback function pointer that will be used in this function.

`*tree_client_data`

Tree client data that will be used in this function.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrObject::ffrReadScopeVarTree

ffrReadVarByLogUOff

Interface

```
fsdbRC ffrObject::ffrReadVarByLogUOff (fsdbLUOff *var_start_log_uoff);
```

Purpose

Read all signals of a scope.

In the tree callback function, when the callback type is **FSDB_TREE_CBT_SCOPE**, the callback data contains a var start logical unsigned offset (*var_start_log_uoff*) which represents the logical start offset of the signals belonging to that scope. Usually, the application has to keep the *var_start_log_uoff* of a scope so that it can read all signals of that scope later.

When the application needs to read all signals of a scope, it should call this function and supply it with the *var_start_log_uoff* of that scope which it kept previously. In this way, all signals of that scope are read from the FSDB file and the tree callback function is activated for each signal.

When the tree callback function is activated, *ffrReadVarByLogUoff()* returns all callback types, except the following scope operation of callback types: **FSDB_TREE_CBT_BEGIN_TREE**, **FSDB_TREE_CBT_SCOPE**, **FSDB_TREE_CBT_UPSCOPE**, **FSDB_TREE_CBT_END_TREE**, **FSDB_TREE_CBT_END_ALL_TREE**.

The application must cast the callback data to **fsdbTreeCBDataVar** in order to get the information of the returned signal. After the function returns, all signals of the desired scope are retrieved.

The typical flow is that the application calls **ffrObject::ffrReadScopeTree** or **ffrObject::ffrReadScopeTree2** to get the *var_start_log_uoff* of all scopes in the FSDB file. It has to keep the *var_start_log_uoff* in its own memory in the meantime. When it needs to retrieve the information of the signals of a certain scope, it calls this function with the kept *var_start_log_uoff* to get the information.

Arguments

**var_start_log_uoff*

Start logical unsigned offset of the signals belonging to the desired scope.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

`ffrObject::ffrReadScopeTree`

`ffrObject::ffrReadScopeTree2`

ffrReadVarByLogUOff

Interface

```
fsdbRC ffrObject::ffrReadVarByLogUOff (fsdbLUOff *var_start_log_uoff, ushort_T  
real_file_obj_idx);
```

Purpose

This API is for virtual files only. When the application needs to read all signals of a scope, it calls this function and supplies it with the *var_start_log_uoff* argument of that scope that was kept previously. Therefore, all signals of that scope will be read from the FSDB file and the tree callback function is activated for each signal. When the tree callback function is activated, the callback type will be **FSDB_TREE_CBT_VAR**. The application should cast the callback data to **fsdbTreeCBDataVar** to get the information of the returned signal. After the function returns, all signals of the desired scope will be retrieved.

Arguments

var_start_log_uoff

Start the logical unsigned offset of the signals belonging to the desired scope.

real_file_obj_idx

Specify the real file object index in the virtual file. This information can be obtained from *real_obj_idx* of call back data (i. e. **ffrTreeCBDataScope**, **ffrTreeCBDataUpScope**, and **ffrTreeCBDataBeginTree**).

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

ffrRegisterErrorCBFunc

Interface

```
static void ffrObject::ffrRegisterErrorCBFunc (fsdbMsgCBFunc cb)
```

Purpose

Redirect error messages issued by the FSDB reader to the user defined callback function *cb*. In this way, the FSDB reader cannot show the original error message like

**ERROR* This is an error message*

Instead, the user defined callback function *cb* can decide how to display or handle this error message. After the error message is handled, the FSDB reader will terminate the process since the "error" here means a fatal error and the program could not continue execution any more.

If *cb* is set to **NULL**, the FSDB reader will regain the control to display the error message in the default format.

This change takes effect for all reader objects in the same process since it is a static member function of **ffrObject** and it must be called before any FSDB reader object is created.

Arguments

cb

User defined callback function that will handle all error messages issued by the FSDB reader.

See Also

`ffrObject::ffrRegisterInfoCBFunc`

`ffrObject::ffrRegisterWarnCBFunc`

ffrRegisterInfoCBFunc

Interface

`static void ffrObject::ffrRegisterInfoCBFunc (fsdbMsgCBFunc cb)`

Purpose

Redirect informative messages issued by the FSDB reader to the user defined callback function *cb*. In this way, the FSDB reader won't show the original informative message like

This is an informative message

Instead, the user defined callback function *cb* can decide how to display or handle this informative message.

If *cb* is set to NULL, the FSDB reader will regain the control to display the informative message in the default format.

This change takes effect for all reader objects in the same process since it is a static member function of **ffrObject** and it must be called before any FSDB reader object is created.

Arguments

cb

User defined callback function that will handle all informative messages issued by the FSDB reader.

See Also

`ffrObject::ffrRegisterWarnCBFunc`

`ffrObject::ffrRegisterErrorCBFunc`

ffrRegisterWarnCBFunc

Interface

```
static void ffrObject::ffrRegisterWarnCBFunc (fsdbMsgCBFunc cb)
```

Purpose

Redirect warning messages issued by the FSDB reader to the user defined callback function *cb*. In this way, the FSDB reader will not show the original warning message like

**WARN* This is a warning message*

Instead, the user defined callback function *cb* can decide how to display or handle this warning message.

If *cb* is set to NULL, the FSDB reader will regain the control to display the warning message in the default format.

This change takes effect for all reader objects in the same process since it is a static member function of **ffrObject** and it must be called before any the FSDB reader object is created.

Arguments

cb

User defined callback function that will handle all warning messages issued by the FSDB reader.

See Also

ffrObject::ffrRegisterInfoCBFunc

ffrObject::ffrRegisterErrorCBFunc

ffrResetSignalList

Interface

```
fsdbRC ffrObject::ffrResetSignalList();
```

Purpose

Resets signal list of the reader object (i.e. empties the signal list so that it contains no signal). To add a signal to the signal list, call **ffrObject::ffrAddToSignalList**.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrObject::ffrAddToSignalList ffrObject::IsInSignalList

ffrResetViewWindow

Interface

```
fsdbRC ffrObject::ffrResetViewWindow (fsdbXTag *start_xtag, fsdbXTag *close_xtag);
```

Purpose

Similar to **ffrSetViewWindow**, this API sets the view window from start_xtag to close_xtag regardless of whether the original view window is set. Refer to **ffrSetViewWindow** for details.

Arguments

**start_xtag*

A pointer to **fsdbXTag** which represents the start_xtag of the view window.

**close_xtag*

A pointer to **fsdbXTag** which represents the close_xtag of the view window.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrObject::ffrGetSessionListInfo

fsdbRC ffrObject::ffrSetViewWindow

ffrSetExitOnMallocFail

Interface

```
static void ffrObject::ffrSetExitOnMallocFail (bool_T flag);
```

Purpose

Notify the FSDB reader if it should terminate the application upon memory allocation failure. The default value is TRUE.

It is recommended not to set the default value to FALSE as it is difficult for the FSDB reader to continue with memory allocation failure. This function should only be used in special circumstances.

This change takes effect for all reader objects in the same process since it is a static member function of **ffrObject**. It can be called at any time to change the behavior of the FSDB reader.

Arguments

flag

TRUE to terminate the application upon memory allocation failure. FALSE to continue execution upon memory allocation failure.

ffrSetFsdbTag64LoadRange

Interface

```
void ffrObject::ffrSetFsdbTag64LoadRange (fsdbTag64 *start, fsdbTag64 *end);
```

Purpose

Set xtag load range. This API is used by Novas only and is obsolete from FSDB file version 2.0.

Arguments

**start*

Start time of the load range.

**end*

End time of the load range.

ffrSetLoadNavDBOnlyOnce

Interface

```
void ffrObject::ffrSetLoadNavDBOnlyOnce (bool_T flag);
```

Purpose

Set if the FSDB reader should always load the navigation database only once.

To make this function work, the navigation database with all needed signals must be loaded the first time. If the navigation database is loaded when the flag is **TRUE**, no more navigation databases are loaded in the future. If the AP tries to load a signal whose navigation database is not loaded initially, the results are wrong.

The navigation database is not unloaded until the FSDB file is closed.

This API is used from FSDB version 2.3.

Arguments

flag

TRUE to turn on loading the navigation database only once. FALSE to turn off loading the navigation database only once.

ffrSetOEMActiveObj

Interface

```
static void ffrObject::ffrSetOEMActiveObj (ffrObject *obj);
```

Purpose

Set *obj* as OEM active object in the FSDB reader. *obj* is kept in the FSDB reader and gets returned when **ffrObject::ffrGetOEMActiveObj** is called.

When this function is invoked, *oem_set_file* in OEM function table is also invoked if it is defined. This API is used from FSDB version 2.3.

OEM active object is set to the FSDB reader itself when **ffrObject::AddToSignalList** is called.

Arguments

**obj*

The FSDB reader object that is to be set to OEM active object.

See Also

ffrObject::ffrGetOEMActiveObj

ffrSetTreeCBFunc

Interface

```
fsdbRC ffrObject::ffrSetTreeCBFunc (fsdbTreeCBFunc tree_cb_func, void  
*tree_client_data);
```

Purpose

Set the value of tree callback function and tree client data of the FSDB reader object. If *tree_client_data* is NULL, the corresponding data in the FSDB reader object will be set to the reader itself.

Arguments

tree_cb_func

Tree callback function pointer.

**tree_client_data*

The tree client data which will be passed back when tree callback function is called.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrObject::ffrGetTreeCBFunc

ffrSetViewWindow

Interface

```
fsdbRC ffrObject::ffrSetViewWindow (fsdbXTag *start_xtag, fsdbXTag *close_xtag);
```

Purpose

Set the view window for partial load of value changes. The x coordinate can be 32-bit or 64-bit integer, float or double, so in general, we call it as xtag. The view window is specified by xtag range: a start xtag and a close xtag. Once it is set, only the value changes fallen in that range are loaded onto memory. This API is used from FSDB version 2.0.

It is recommended that application specifies the start_xtag of view window as the start_xtag of a certain flush session and close_xtag of view window as the close_xtag of a certain flush session. The reason is that when loading value changes, the FSDB reader always loads a complete flush session. Therefore, the start_xtag/close_xtag of flush sessions could be used as a hint of the width of a view window. For the concept of a flush session, see **ffrObject::ffrGetSessionListInfo** for details.

Arguments

`*start_xtag`

A pointer to **fsdbXTag** which represents the start xtag of the view window.

`*close_xtag`

A pointer to **fsdbXTag** which represents the close xtag of the view window.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

`ffrObject::ffrGetSessionListInfo`

ffrSync

Interface

`fsdbRC ffrObject::ffrSync (bool_T &is_file_complete);`

Purpose

Detect if the FSDB file has new data coming in under interactive mode and read it if any. This function is necessary only when application needs to deal with the FSDB file under interactive mode.

New data includes newly added design hierarchies, value changes, data type definitions, and dump-off ranges.

Arguments

`is_file_complete`

Indicate if the FSDB file has finished or not.

If the FSDB file has not yet finished when the function returns, the argument *is_file_complete* will be **FALSE**. If it has finished, *is_file_complete* will be **TRUE**. Referring to the argument can avoid unnecessary calling of **ffrObject::frSync** when the FSDB file has finished.

Return Value

FSDB_RC_SUCCESS if there is new data coming in. **FSDB_RC_FAILURE** if:

- There is no new data coming in.
- The FSDB file has finished.
- Errors occur in previous operations.

ffrUnloadSignals

Interface

```
fsdbRC ffrObject::ffrUnloadSignals();
```

Purpose

Unload value changes that are already loaded up to now.

Signals have reference count from FSDB file version 2.0. If a signal's reference count is not zero, its value changes will not be unloaded.

The reference count of a signal is increased by 1 if a value change traverse handle is created on it by **ffrObject::ffrCreateVCTrvsHdl**. It is decreased by 1 if a previously created value change traverse handle is freed by **ffrVClterOne::ffrFree**. **ffrVClterOne::ffrGetRefCount** returns current reference count of a signal. Signal's reference count is added from FSDB file version 2.0.

Return Value

FSDB_RC_SUCCESS if successful. **FSDB_RC_IN_USE** if the reference count of any signal is not zero and the value changes of the signal could not be unloaded.

See Also

`ffrObject::ffrCreateVCTrvsHdl`

`ffrVClterOne::ffrFree`

ffrVCIterOne::ffrGetRefCount

ffrUnloadSignals

Interface

```
fsdbRC ffrObject::ffrUnloadSignals (fsdbVarIdcode var_idcode_arr[], fsdbVarCount count);
```

Purpose

Unload the value changes of the signals in *var_idcode_arr* from memory.

Signals have reference count from FSDB file version 2.0. If a signal's reference count is not zero, its value changes will not be unloaded.

The reference count of a signal is increased by 1 if a value change traverse handle is created on it by **ffrObject::ffrCreateVCTrvsHdl**. It is decreased by 1 if a previously created value change traverse handle is freed by **ffrVCIterOne::ffrFree**. **ffrVCIterOne::ffrGetRefCount** returns current reference count of a signal. Signal's reference count is added from FSDB file version 2.0.

Arguments

var_idcode_arr

The idcode array of the signals whose value changes are to be unloaded.

count

The number of signals in *var_idcode_arr*. For 32-bit platform, the type is `uint_T`. For 64-bit platform, the type is `longlong_T`.

Return Value

FSDB_RC_SUCCESS if successful. **FSDB_RC_IN_USE** if the reference count of any of the signals in *var_idcode_arr* is not zero and the value changes of the signal could not be unloaded.

See Also

ffrObject::ffrCreateVCTrvsHdl

ffrVCIterOne::ffrFree

ffrVCIterOne::ffrGetRefCount

ffrUnloadSignals

Interface

```
fsdbRC ffrObject::ffrUnloadSignals (fsdbVarIdcode var_idcode);
```

Purpose

Unload the value changes of the signal specified by *var_idcode* from memory.

Signals have reference count from FSDB file version 2.0. If a signal's reference count is not zero, its value changes will not be unloaded.

The reference count of a signal is increased by 1 if a value change traverse handle is created on it by **ffrObject::ffrCreateVCTrvsHdl**. It is decreased by 1 if a previously created value change traverse handle is freed by **ffrVCIterOne::ffrFree**. **ffrVCIterOne::ffrGetRefCount** returns current reference count of a signal.

Arguments

var_idcode

The idcode of the signal whose value changes are to be unloaded.

Return Value

FSDB_RC_SUCCESS if successful. **FSDB_RC_FAILURE** if *var_idcode* is invalid.

FSDB_RC_IN_USE if the reference count of signal is not zero and the value changes of the signal can not be unloaded.

See Also

`ffrObject::ffrCreateVCTrvsHdl`

`ffrVCIterOne::ffrFree`

`ffrVCIterOne::ffrGetRefCount`

ffrWarnSuppress

Interface

```
static void ffrObject::ffrWarnSuppress (bool_T if_suppress);
```

Purpose

Suppress or turn on all warning messages issued by the FSDB reader objects. This API is used from FSDB version 2.3.

This change takes effect for all reader objects in the same process since it is a static member function of **ffrObject** and it must be called before any FSDB reader object is created.

Arguments

`if_suppress`

TRUE if all warning messages need to be suppressed; FALSE otherwise.

See Also

`ffrObject::ffrRegisterWarnCBFunc`

ffrTimeBasedVCIterOne

ffrFree

Interface

```
void ffrTimeBasedVCIterOne::ffrFree();
```

Purpose

Free the time-based value change traverse object.

It is similar to **ffrVCIterOne::ffrFree**. See **ffrVCIterOne::ffrFree** for details.

See Also

`ffrObject::ffrCreateTimeBasedVCTrvsHdl`

`ffrVCIterOne::ffrFree`

ffrGetBitSize

Interface

```
ushort_T ffrTimeBasedVCIterOne::ffrGetBitSize();
```

Purpose

When the traverse handle stops at certain value change of certain signal, get the bit size of the signal. It is similar to **ffrVCIterOne::ffrGetBitSize**. See **ffrVCIterOne::ffrGetBitSize** for details.

Return Value

Bit size of the signal if successful; **USHRT_MAX** otherwise.

See Also

`ffrVCIterOne::ffrGetBitSize`

`ffrTimeBasedVCIterOne::ffrGetByteCount`

`ffrTimeBasedVCIterOne::ffrGetBytesPerBit`

ffrGetByteCount

Interface

```
uint_T ffrTimeBasedVCIterOne::ffrGetByteCount();
```

Purpose

When the traverse handle stops at certain value change of certain signal, get the byte count of the signal. It is similar to **ffrVCIterOne::ffrGetByteCount**. See **ffrVCIterOne::ffrGetByteCount** for details.

Return Value

Byte count of the signal if successful; **UINT_MAX** otherwise.

See Also

`ffrVCIterOne::ffrGetByteCount`

`ffrTimeBasedVCIterOne::ffrGetBitSize`

`ffrTimeBasedVCIterOne::ffrGetBytesPerBit`

ffrGetBytesPerBit

Interface

```
fsdbBytesPerBit ffrTimeBasedVCIterOne::ffrGetBytesPerBit();
```

Purpose

When the traverse handle stops at certain value change of certain signal, get the bytes per bit of the signal. It is similar to **ffrVCIterOne::ffrGetBytesPerBit**. See **ffrVCIterOne::ffrGetBytesPerBit** for details.

Return Value

Bytes per bit of the signal if successful; **FSDB_BYTES_PER_BIT_1B** otherwise.

ffrGetGlitchNum

Interface

```
fsdbRC ffrTimeBasedVCIterOne::ffrGetGlitchNum (int *ret_glitch_num);
```

Purpose

When the traverse handle stops at certain value change of certain signal, get the number of glitches this signal has at that time step. It is similar to **ffrVCIterOne::ffrGetGlitchNum**. See **ffrVCIterOne::ffrGetGlitchNum** for details.

Arguments

ret_glitch_num

A pointer to an integer which represents the number of glitches.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

`ffrVCIterOne::ffrGetBytesPerBit`

`ffrTimeBasedVCIterOne::ffrGetBitSize`

`ffrTimeBasedVCIterOne::ffrGetByteCount`

ffrGetSeqNum

Interface

```
fsdbRC ffrTimeBasedVCIterOne::ffrGetSeqNum (fsdbSeqNum *ret_seq_num);
```

Purpose

When the traverse handle stops at certain value change of certain signal, get the sequence number of the value change of the signal. It is similar to **ffrVCIterOne::ffrGetSeqNum**. See **ffrVCIterOne::ffrGetSeqNum** for other details.

Arguments

`*ret_seq_num`

A pointer to **fsdbSeqNum** representing the sequence number. **fsdbSeqNum** is defined in **fsdbShr.h** header file.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

`ffrVCIterOne::ffrGetSeqNum` `ffrTimeBasedVCIterOne::ffrGetVarIdcodeXTagVCSeqNum`

ffrGetVarIdcode

Interface

`fsdbRC ffrTimeBasedVCIterOne::ffrGetVarIdcode (fsdbVarIdcode *ret_var_idcode);`

Purpose

When the traverse handle stops at certain value change of certain signal, get idcode of the signal.

Arguments

`*ret_var_idcode`

Idcode of the signal. **FSDB_INVALID_VAR_IDCODE** if the traverse handle does not point to any value change.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

`ffrTimeBasedVCIterOne::ffrGetVarIdcodeXTagVCSeqNum`

ffrGetVarIdcodeXTagVCSeqNum

Interface

`fsdbRC ffrTimeBasedVCIterOne::ffrGetVarIdcodeXTagVCSeqNum (fsdbVarIdcode *var_idcode, fsdbXTag *xtag, byte_T **vc, fsdbSeqNum *seq_num);`

Purpose

When the traverse handle stops at certain value change of certain signal, get idcode, xtag, value, and sequence number of the signal. It is similar to **ffrVCIterOne::ffrGetXTagVCSeqNum**. See **ffrVCIterOne::ffrGetXTagVCSeqNum** for details.

Arguments

`*var_idcode`

A pointer to **fsdbVarIdcode** representing the returned idcode of the var.

`*xtag`

A pointer to **fsdbXTag** representing the returned xtag.

`**vc`

A pointer to `byte_T*` representing the value.

`*seq_num`

A pointer to **fsdbSeqNum** representing the sequence number. **fsdbSeqNum** is defined in **fsdbShr.h**.

Return Value

FSDB_RC_SUCCESS if successful. This function returns **FSDB_RC_FAILURE** when the traverse handle does not point to any value change or the FSDB version is earlier than 2.0.

See Also

`ffrTimeBasedVCIterOne::ffrGetVarIdcode`

`ffrTimeBasedVCIterOne::ffrGetXTag`

`ffrTimeBasedVCIterOne::ffrGetVC`

`ffrTimeBasedVCIterOne::ffrGetSeqNum`

ffrGetVarType

Interface

`fsdbVarType ffrTimeBasedVCIterOne::ffrGetVarType();`

Purpose

When the traverse handle stops at certain value change of certain signal, get var type of the signal. **fsdbVarType** is defined in **fsdbShr.h** header file. It is similar to **ffrVCIterOne::ffrGetVarType**. See **ffrVCIterOne::ffrGetVarType** for details.

Return Value

Var type of the signal if successful; 0 otherwise.

See Also

ffrVCIterOne::ffrGetVarType

ffrGetVC

Interface

```
fsdbRC ffrTimeBasedVCIterOne::ffrGetVC (byte_T **ret_vc);
```

Purpose

When the traverse handle stops at certain value change of certain signal, get value change of the signal. It is similar to **ffrVCIterOne::ffrGetVC**. See **ffrVCIterOne::ffrGetVC** for details.

Arguments

`**ret_vc`

A pointer to byte_T* representing the value change.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrTimeBasedVCIterOne::ffrGetVC

ffrVCIterOne::ffrGetVarIdcodeXTagVCSeqNum

ffrGetXTag

Interface

```
fsdbRC ffrTimeBasedVCIterOne::ffrGetXTag (void *ret_xtag);
```

Purpose

When the traverse handle stops at certain value change of certain signal, get xtag of the value change this traverse handle points to now. It is similar to **ffrVCIterOne::ffrGetXTag**. See **ffrVCIterOne::ffrGetXTag** for details.

Arguments

*ret_xtag

A pointer to void representing returned xtag. It must be cast to **fsdbTag64*** to get the actual value.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrVCIterOne::ffrGetXTag

ffrVCIterOne::ffrGetVarIdcodeXTagVCSeqNum

ffrGotoNextVC

Interface

```
fsdbRC ffrTimeBasedVCIterOne::ffrGotoNextVC();
```

Purpose

Point the traverse handle to next value change. Since this is a time-based traverse handle, the value change may belong to another signal. Application must notice this.

The function is similar to **ffrVCIterOne::ffrGotoNextVC**. See **ffrVCIterOne::ffrGotoNextVC** for details.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrVCIterOne::ffrGotoNextVC

ffrlsItPropVar

Interface

```
bool_T ffrTimeBasedVCIterOne::ffrlsItPropVar() const;
```

Purpose

When the traverse handle stops at certain value change of certain signal, return if the signal is a property signal or not. This API is used from FSDB file version 2.3.

Return Value

TRUE if the signal is a property signal; FALSE otherwise or the FSDB file version is older than 2.3.

ffrlsItStreamVar

Interface

```
bool_T ffrTimeBasedVCIterOne::ffrlsItStreamVar() const;
```

Purpose

When the traverse handle stops at certain value change of certain signal, return if the signal is a stream signal or not. This API is used from FSDB file version 2.3.

Return Value

TRUE if the signal is a stream signal; FALSE otherwise or the FSDB file version is earlier than 2.3.

ffrVCIterOne

ffrFree

Interface

```
void ffrVCIterOne::ffrFree();
```

Purpose

Free the value change traverse object.

When the traverse object is freed, the reference count of the signal it is associated with will be decreased by one. Only when the reference count of a signal is zero could its value changes be unloaded by **ffrObject::ffrUnloadSignals**. Hence it is import to free a traverse object when it is no longer in use.

This function does not unload the value changes associated with the signal. It just frees the traverse object itself. Call **ffrObject::ffrUnloadSignals** to actually unload a signal's value changes

See Also

ffrObject::ffrCreateVCTrvsHdl

ffrVCIterOne::ffrGetRefCount

ffrObject::ffrUnloadSignals

ffrGetBitSize

Interface

```
ushort_T ffrVCIterOne::ffrGetBitSize();
```

Purpose

Get the bit size of the signal this traverse object is associated with. Bit size is the width of a signal. For example, the bit size of the signal *out* below:

```
reg [3:0] out;  
is 4.
```

Return Value

Bit size of the signal if successful; **USHRT_MAX** otherwise.

See Also

ffrVCIterOne::ffrGetByteCount

ffrVCIterOne::ffrGetBytesPerBit

ffrGetByteCount

Interface

```
uint_T ffrVCIterOne::ffrGetByteCount();
```

Purpose

Get the byte count of the signal this traverse object is associated with. The definition of byte count is

```
Byte count = Bit size * Bytes per bit.
```

See **ffrVCIterOne::ffrGetBitSize** for the definition of bit size and **ffrVCIterOne::ffrGetBytesPerBit** for the definition of bytes per bit.

Return Value

Byte count of the signal if successful; **UINT_MAX** otherwise.

See Also

ffrVCIterOne::ffrGetBitSize

ffrVCIterOne::ffrGetBytesPerBit

ffrGetBytesPerBit

Interface

```
fsdbBytesPerBit ffrVCIterOne::ffrGetBytesPerBit();
```

Purpose

Get the bytes per bit of the signal.

Bytes per bit of a signal is the number of bytes needed in the FSDB reader to store a bit of the signal. For example, in the Verilog code below, bytes per bit of a is 1 since it's a wire and counter is 4 since it's a 32-bit integer.

```
wire a;  
integer counter;
```

Return Value

Bytes per bit of the signal if successful; **FSDB_BYTES_PER_BIT_1B** otherwise.

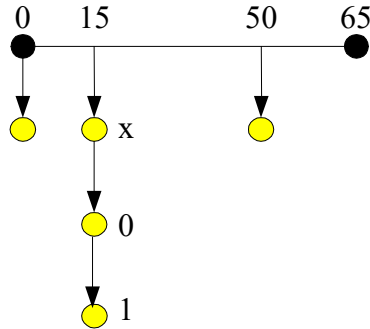
ffrGetGlitchNum

Interface

```
fsdbRC ffrVCIterOne::ffrGetGlitchNum (int *ret_glitch_num);
```

Purpose

Get the number of glitches this signal has at current time step.



In the example above, if the traverse handle points to a value change (x, 0, or 1) at time 15, this function returns 3 since the signal has 3 glitches at that time.

Arguments

`*ret_glitch_num`

A pointer to an integer which represents the number of glitches.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

ffrGetMaxXTag

Interface

```
fsdbRC ffrVCIterOne::ffrGetMaxXTag (void *ret_xtag);
```

Purpose

Get the xtag of the last value change of the signal.

The value of `ret_xtag` is affected by view window. That is, if the view window is from 100 to 1000, `ret_xtag` will be 1000 even if the signal has other value changes that occur at larger time steps in the FSDB file.

Arguments

`*ret_xtag`

A pointer to void representing the xtag of the last value change. It must be cast to **fsdbTag64*** to get the actual value.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrObject::ffrSetViewWindow

ffrVCIterOne::ffrGetMinXTag

ffrGetMinXTag

Interface

```
fsdbRC ffrVCIterOne::ffrGetMinXTag (void *ret_xtag);
```

Purpose

Get the xtag of the first value change of the signal.

The value of *ret_xtag* is affected by view window. If the view window is from 100 to 1000, *ret_xtag* will be 100 even if the signal has other value changes that occur at smaller time steps in the FSDB file.

Arguments

**ret_xtag*

A pointer to void representing the xtag of the first value change. It must be casted to **fsdbTag64*** to get the actual value.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrObject::ffrSetViewWindow

ffrVCIterOne::ffrGetMaxXTag

ffrGetRefCount

Interface

```
uint_T ffrVCIterOne::ffrGetRefCount() const;
```

Purpose

Get current reference count of the signal.

The reference count of a signal is increased by 1 if a value change traverse handle is created on it by **ffrObject::ffrCreateVCTrvsHdl**. It is decreased by 1 if a previously created value change traverse handle is freed by **ffrVCIterOne::ffrFree**. Signal's reference count is added from FSDB file version 2.0.

Return Value

The reference count of the signal.

See Also

ffrObject::ffrCreateVCTrvsHdl

ffrVCIterOne::ffrFree

ffrGetSeqNum

Interface

```
fsdbRC ffrVCIterOne::ffrGetSeqNum (fsdbSeqNum *ret_seq_num);
```

Purpose

Get sequence number of the value change this traverse handle points to now.

Application should call **ffrObject::ffrIsThereSeqNum** to detect if the FSDB file contains sequence number or not first. If the FSDB file does not contain sequence number and applications calls this function, the value of **ret_seq_num* will be 0.

Arguments

**ret_seq_num*

A pointer to **fsdbSeqNum** representing the sequence number. **fsdbSeqNum** is defined in **fsdbShr.h** header file.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrObject::ffrIsThereSeqNum

ffrVCIterOne::ffrGetXTagVCSeqNum

ffrGetTransInfo

Interface

```
fsdbRC ffrVCIterOne::ffrGetTransInfo (ffrTransInfo *&info);
```

Purpose

Gets the transaction value this traverse handle points to now. The result will be filled in *info* by the FSDB reader. See **ffrVCIterOne::ffrGetTransInfo** for details.

The application must make sure the signal is a stream signal before calling this function; otherwise, unexpected results may occur. Also, before calling this API, application must call **ffrObject::ffrReadDataTypeDefByBlkIdx** to read the definition of attributes; otherwise, unexpected results or crashes may occur.

The API is used from FSDB file version 2.0.

Arguments

**info*

An **ffrTransInfo** that will hold the result returned by the FSDB reader. See **ffrTransInfo** in **Transaction Traversal** section for details.

Return Value

FSDB_RC_SUCCESS on success; **FSDB_RC_FAILURE** otherwise.

See Also

ffrObject::ffrGetTransInfo

ffrGetTransVal

Interface

```
fsdbRC ffrVCIterOne::ffrGetTransVal (fsdbTransId &trans_id, str_T &label, fsdbXTag  
&begin_time, fsdbXTag &end_time, fsdbTransStatus &status, ushort_T &attr_count,  
ushort_T *&attr_hdl_arr, byte_T **&attr_val_arr, fsdbSeqNum &seq_num);
```

Purpose

This function is used to get the transaction values of streams just like **ffrVCIterOne::ffrGetVC** is used to get value change of ordinary signals. Application must use **ffrObject::ffrCreateVCTrvsHdl** to create a traverse handle of a stream before calling this function.

Before calling this API, application must call **ffrObject::ffrReadDataTypeDefByBlkIdx** to read the definition of attributes; otherwise, unexpected results or crashes may occur.

This API has the same usage as **ffrVCIterOne::ffrGetTransInfo**. It is suggested to replace this API with **ffrVCIterOne::ffrGetTransInfo**.

This API is used from FSDB file version 2.0.

Arguments

```
trans_id, label, begin_time, end_time, status, attr_count, *attr_hdl_arr,  
**attr_val_arr
```

See **ffrTransInfo** in **Transaction Traversal** section for details.

`seq_num`

A reference to the sequence number of the transaction. It is meaningless in most cases since a transaction is a combination of `begin_transaction/end_transaction` and `seq_num` is just the value of `begin_transaction`. It is already obsolete in **ffrTransInfo**.

Return Value

FSDB_RC_SUCCESS on success; **FSDB_RC_FAILURE** otherwise.

ffrGetVarType

Interface

```
fsdbVarType ffrVCIterOne::ffrGetVarType();
```

Purpose

Get var type of the signal. **fsdbVarType** is defined in **fsdbShr.h** header file.

Return Value

Var type of the signal if successful; 0 otherwise.

ffrGetVC

Interface

```
fsdbRC ffrVCIterOne::ffrGetVC (byte_T **ret_vc);
```

Purpose

Get value of the value change this traverse handle points to now.

The FSDB reader will point **ret_vc* to a local buffer. Since the buffer can be modified any time, application should copy the buffer to its own memory if it is to keep the value and use it later.

Arguments

****ret_vc**

A pointer to to byte_T* representing the value change.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrVCIterOne::ffrGetXTagVCSeqNum

ffrGetXTag

Interface

fsdbRC ffrVCIterOne::ffrGetXTag (void **ret_xtag*);

Purpose

Get xtag of the value change this traverse handle points to now. Usually, this function is called after **ffrVCIterOne::ffrGotoXTag** is made successfully.

Arguments

***ret_xtag**

A pointer to void representing returned xtag. It must be casted to **fsdbTag64*** to get the actual value.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrVCIterOne::ffrGetXTagVCSeqNum

ffrGetXTagVCSeqNum

Interface

```
fsdbRC ffrVCIterOne::ffrGetXTagVCSeqNum (fsdbXTag *xtag, byte_T **vc, fsdbSeqNum *seq_num);
```

Purpose

Get xtag, value, and sequence number of the value change this traverse handle points to now.

If an application needs to retrieve the value of *seq_num*, it should call **ffrObject::ffrIsThereSeqNum** to detect if the FSDB file contains sequence number or not first. If the FSDB file does not contain sequence number and applications calls this function, the value of **seq_num* will be 0.

This function only supports from FSDB version 2.0.

Arguments

**xtag*

A pointer to **fsdbXTag** representing the returned xtag.

***vc*

A pointer to *byte_T** representing the value.

**seq_num*

A pointer to **fsdbSeqNum** representing the sequence number. **fsdbSeqNum** is defined in **fsdbShr.h**.

Return Value

FSDB_RC_SUCCESS if successful. This function returns **FSDB_RC_FAILURE** when the signal does not have any value change or the FSDB file version is older than 2.0.

See Also

ffrVCIterOne::ffrGetXTag

ffrVCIterOne::ffrGetVC

ffrVCIterOne::ffrGetSeqNum

ffrObject::IsThereSeqNum

ffrGotoNextVC

Interface

```
fsdbRC ffrVCIterOne::ffrGotoNextVC();
```

Purpose

Point value change traverse handle to next value change.

If the traverse handle already points to the last value change, this function returns **FSDB_RC_FAILURE**.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrVCIterOne::ffrGotoPrevVC

ffrGotoPrevVC

Interface

```
fsdbRC ffrVCIterOne::ffrGotoPrevVC();
```

Purpose

Point value change traverse handle to previous value change.

If the traverse handle already points to the first value change, this function returns **FSDB_RC_FAILURE**.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrVCIterOne::ffrGotoNextVC

ffrGotoTheFirstVC

Interface

```
fsdbRC ffrVCIterOne::ffrGotoTheFirstVC();
```

Purpose

Point the value change traverse handle to the first value change.

This function succeeds even if the corresponding signal does not have any value change. However, subsequent operations such as **ffrVCIterOne::ffrGetVC**, **ffrVCIterOne::ffrGetXTag**, and **ffrVCIterOne::ffrGetSeqNum** and so on will fail.

Return Value

This function always returns **FSDB_RC_SUCCESS**.

ffrGotoXTag

Interface

```
fsdbRC ffrVCIterOne::ffrGotoXTag (void *xtag, int *ret_glitch_num = NULL);
```

Purpose

Point the value change traverse handle to the last value change at *xtag*.

If there is no value change at *xtag*, the value change traverse handle is aligned based on the jump behavior defined by the FSDB reader and *xtag* will be updated based on the aligned value change. For details, refer to **Jump Behavior** section.

If there is (are) value change(s) at *xtag*, the traverse handle is moved to the last value change at *xtag* and the number of glitches at *xtag* is calculated and returned to the application.

Argument

**xtag*: The specified x co-ordinate point.

Return Value

FSDB_RC_SUCCESS if successful; **FSDB_RC_FAILURE** otherwise.

See Also

ffrVCIterOne::ffrGotoTheFirstVC

ffrHasIncoreVC

Interface

```
bool_T ffrVCIterOne::ffrHasIncoreVC();
```

Purpose

Check the signal to see if it has any value change or not. Usually, this function is used when the FSDB file is not finished and still growing.

Return Value

TRUE if the signal has value changes; FALSE otherwise.

Other

void ffrOEMInit

Interface

```
void ffrOEMInit (int argc, char *argv[], void *xt_ap_context, ffrOEMFuncTbl  
*ret_oem_fun_tbl, ffrInsertOEMComputedVC insert_oem_computed_vc);
```

Purpose

OEM can compute value changes and insert the result into Novas so that the result could be displayed on the waveform window. **ffrOEMInit** is used to initialize this scheme. It is called inside Novas, not in OEM products.

After this function is called, when actions defined in *ret_oem_fun_tbl* is invoked in Novas, OEM-defined callback functions are called and they can respond to these actions. *insert_oem_computed_vc* is used to insert value changes into the FSDB reader memory.

Whether invoking this scheme or not is determined when Novas is built. It is not invoked in normal releases of Novas. OEM needs to cooperate with Novas for this scheme to work.

Deprecated

This function has been replaced by **ffrOEMInit2()**. Although the **ffrOEMInit** function still works, it will not be enhanced in the future.

Arguments

argc

The number of command line arguments when Novas is invoked.

**argv*

The command line arguments when Novas is invoked.

**xt_ap_context*

An `XtAppContext` structure passed to the OEM.

`*ret_oem_func_tbl`

An OEM function table kept by the FSDB reader. When Novas calls **ffrOEMInit**, it passes this function table to the OEM so that the OEM could fill its content with their own functions. See **ffrOEMFuncTbl** in **ffrAPI.h** to see what functions are available.

`insert_oem_computed_vc`

An OEM function kept by the FSDB reader. When Novas calls **ffrOEMInit**, it passes this function to the OEM so that the OEM could insert computed value changes into the FSDB reader memory.

See Also

`ffrOEMInit2`

void ffrOEMInit2

Interface

```
void ffrOEMInit2 (int argc, char *argv[], void *xt_app_context, ffrOEMFuncTbl  
*ret_oem_func_tbl, ffrToOEMFuncTbl *ret_to_oem_func_tbl);
```

Purpose

This function is similar to **ffrOEMInit** with the difference that `insert_oem_computed_vc` is replaced with `ret_to_oem_func_tbl` so that is easier to extend. See **ffrOEMInit** for details.

Arguments

`argc`

The number of command line arguments when Novas is invoked.

`*argv`

The command line arguments when Novas is invoked.

`*xt_ap_context`

An `XtAppContext` structure passed to the OEM.

`*ret_oem_func_tbl`

An OEM function table kept by the FSDB reader. When Novas calls **ffrOEMInit**, it passes this function table to the OEM so that the OEM could fill its content with their own functions. See **ffrOEMFuncTbl** in **ffrAPI.h** to see what functions are available.

`*ret_to_oem_func_tbl`

An FSDB reader function table passed to OEM. When Novas calls **ffrOEMInit2**, it passes this table to the OEM so that the OEM could perform the operations FSDB reader provides such as inserting computed value changes into the FSDB reader memory.

See Also

ffrOEMInit

3

Mapping Between Type and Data

The following table shows mapping between tree callback type and tree callback data:

Tree Callback Type	Tree Callback Data Type
FSDB_TREE_CBT_BEGIN_TREE	fsdbTreeCBDataBeginTree
FSDB_TREE_CBT_SCOPE	fsdbTreeCBDataScope
FSDB_TREE_CBT_VAR	fsdbTreeCBDataVar
FSDB_TREE_CBT_UPSCOPE	fsdbTreeCBDataUpscope
FSDB_TREE_CBT_END_TREE	fsdbTreeCBDataEndTree
FSDB_TREE_CBT_END_ALL_TREE	N/A
FSDB_TREE_CBT_DT_ENUM	fsdbTreeCBDataEnum
FSDB_TREE_CBT_DT_INT	fsdbTreeCBDataInt
FSDB_TREE_CBT_DT_FLOAT	fsdbTreeCBDataFloating
FSDB_TREE_CBT_DT_FLOATING	fsdbTreeCBDataFloating
FSDB_TREE_CBT_DT_PHYSICAL_BEGIN	fsdbTreeCBDataPhyBegin
FSDB_TREE_CBT_DT_PHY_BEGIN	fsdbTreeCBDataPhyBegin
FSDB_TREE_CBT_DT_PHYSICAL_SECONDARY_UNIT	fsdbTreeCBDataPhySecUnit
FSDB_TREE_CBT_DT_PHY_SEC_UNIT	fsdbTreeCBDataPhySecUnit
FSDB_TREE_CBT_DT_PHYSICAL_END	N/A
FSDB_TREE_CBT_DT_PHY_END	N/A
FSDB_TREE_CBT_DT_INT_H_N_L	fsdbTreeCBDataIntHnL
FSDB_TREE_CBT_ARRAY_BEGIN	fsdbTreeCBDataArrayBegin
FSDB_TREE_CBT_ARRAY_END	N/A

Tree Callback Type	Tree Callback Data Type
FSDB_TREE_CBT_RECORD_BEGIN	fsdbTreeCBDataRecordBegin
FSDB_TREE_CBT_RECORD_END	N/A
FSDB_TREE_CBT_BUS_BEGIN	fsdbTreeCBDataBusBegin
FSDB_TREE_CBT_BUS_ELEMENT	fsdbTreeCBDataBusElement
FSDB_TREE_CBT_BUS_END	N/A
FSDB_TREE_CBT_FILE_TYPE	N/A
FSDB_TREE_CBT_SIMULATOR_VERSION	char*
FSDB_TREE_CBT_SIMULATION_DATE	char*
FSDB_TREE_CBT_X_AXIS_SCALE	char*
FSDB_TREE_CBT_STRUCT_BEGIN	fsdbTreeCBDataStructBegin
FSDB_TREE_CBT_STRUCT_END	N/A
FSDB_TREE_CBT_DT_ATTR_UNKNOWN_TYPE	fsdbTreeCBDataAttr
FSDB_TREE_CBT_DT_ATTR_USER_DEFINED	fsdbTreeCBDataAttr
FSDB_TREE_CBT_DT_ATTR_LOGIC	fsdbTreeCBDataAttr
FSDB_TREE_CBT_DT_ATTR_BOOL	fsdbTreeCBDataAttr
FSDB_TREE_CBT_DT_ATTR_CHAR	fsdbTreeCBDataAttr
FSDB_TREE_CBT_DT_ATTR_STRING	fsdbTreeCBDataAttr
FSDB_TREE_CBT_DT_ATTR_INT32	fsdbTreeCBDataAttr
FSDB_TREE_CBT_DT_ATTR_INT64	fsdbTreeCBDataAttr
FSDB_TREE_CBT_DT_ATTR_UINT32	fsdbTreeCBDataAttr
FSDB_TREE_CBT_DT_ATTR_UINT64	fsdbTreeCBDataAttr
FSDB_TREE_CBT_DT_ATTR_FLOAT	fsdbTreeCBDataAttr
FSDB_TREE_CBT_DT_ATTR_DOUBLE	fsdbTreeCBDataAttr
FSDB_TREE_CBT_DT_ATTR_ENUM	fsdbTreeCBDataEnumAttr

Tree Callback Type	Tree Callback Data Type
FSDB_TREE_CBT_DT_ENUM2	fsdbTreeCBDataEnum2

4

FAQs

As the combined usage of APIs is complex, special usages are collected in this chapter for reference.

Load on Demand

Question 1: In the current Verdi flow, essential signals in a fan-in cone are collected and loaded together. Does this still need to be done in the new FSDB file, or will the FSDB Reader decide which ones to load?

Answer 1: It is not necessary to do so, although it is more efficient. When the value changes of several signals occur in the same block and several signals are loaded at the same time, the block will only be decompressed for a time. If the signals are loaded separately, the block will be decompressed many times. Of course, the former is more efficient.

Question 2: When the simulation time window is large, will the signals be divided by time slots?

Answer 2: No, `ffrSetViewWindow()` must be called to enable partial loading.

Question 3: When time 0 was passed to `ffrGotoXTag`, the memory usage suddenly increased 150 MB. It is a clock signal, so the decompressed size was expected to be large if the entire signal is loaded. Does the new FSDB reader read the entire signal or just a time slot of it?

Answer 3: If the value change of the clock signal was not loaded and `ffrGotoXTag()` was allowed to load them, only the value change at the specified time step is loaded. When the first `LoadSignal` API is called, the FSDB reader will build a navigation database for all signals in the FSDB file if the estimated navigation database size is smaller than 200 MB. The 150 MB most likely comes from here. 200MB is a trade-off number. It speeds up later value change traversal, but it does not slow down the first `LoadSignal` API too much.

Question 4: Clock signals seem to be memory monsters because of their regular value changes. Can clock signals be kept in the compressed format (or a formula to re-calculate waveform), and traversed in a special way?

Answer 4: From past experience, special encoding of clock value changes does not save much space in ordinary FSDB files as clock signals are always 1-bit and an FSDB file

usually does not have many clock signals. Special encoding complicates the FSDB Writer/Reader; therefore, it is not included in FSDB files.

ffrCreateTimeBasedTrvsHdl() & ffrResetViewWindow()

Question: Sometimes it is necessary to rewind to time 0 (and read again until the end, and repeat it again and again), and sometimes it is necessary to rewind to the start of some view window. What is the best sequence of calls to make and parameters to pass? Is it necessary to **ffrFree()** and create a new time-based handle or is there an easier way for rewinding to 0? Is it good enough to have `ffrObject::ffrResetViewWindow()` repeated calls to rewind the FSDB file for re-reading or is using:

```
ffrFree()
ffrCreateTimeBasedVCTrvsHdl(uint_T num, fsdbVarIdcode sig_arr[])
```

better?

Answer: A new time based traverse handle must be re-generated when the view window is changed. For example:

```
ffrSetViewWindow();
ffrCreateTimeBasedTrvsHdl();
// do something
ffrResetViewWindow()
ffrFree();
ffrCreateTimeBasedTrvsHdl();
// do something
...
```

ffrGetVC()

Question: Can a value change pointer returned by **ffrGetVC()** be stored and used later?

Answer: When using **ffrGetVC()** to get value changes of a signal, remember to copy the returned value to the buffer or use it immediately. Do NOT store the returned pointer and use it later. It is not guaranteed that the pointer always points to the correct value change.

For example, if an integer signal has value changes at:

```
(0 0) 32
(0 20) 4567890
```

The following code segment:

```
time.H = 0;
time.L = 0;
vc_trvs_hdl->ffrGotoXTag(&time);
vc_trvs_hdl->ffrGetVC(&vc_ptr);
```

```
fprintf(stdout, "1. [Use It Immediately] Value at (0 0) is %d\n",
*((int*)vc_ptr));
// Copy to a local buffer
memcpy((void*) &int_val1, (void*) vc_ptr, sizeof(int));

time.H = 0;
time.L = 10;
vc_trvs_hdl->ffrGotoXTag(&time);
vc_trvs_hdl->ffrGetVC(&vc_ptr);
// Copy to a local buffer
memcpy((void*) &int_val2, (void*) vc_ptr, sizeof(int));

fprintf(stdout, "1. [Use It Immediately] Value at (0 10) is %d\n",
*((int*)vc_ptr));
fprintf(stdout, "2. [Store the Value and Use It Later] Value at (0 0)
is %d\n", int_val1);
fprintf(stdout, "2. [Store the Value and Use It Later] Value at (0
10) is %d\n", int_val2);
fprintf(stdout, "3. [Store the Pointer and Use It Later] Value at (0
0) is %d\n", *((int*)vc_ptr));
fprintf(stdout, "3. [Store the Pointer and Use It Later] Value at (0
10) is %d\n", *((int*)vc_ptr));
```

may produce the following results:

1. [Use It Immediately] Value at (0 0) is 32
1. [Use It Immediately] Value at (0 10) is 4567890
2. [Store the Value and Use It Later] Value at (0 0) is 32
2. [Store the Value and Use It Later] Value at (0 10) is 4567890
3. [Store the Pointer and Use It Later] Value at (0 0) is 4567890
3. [Store the Pointer and Use It Later] Value at (0 10) is 4567890

If you use it immediately as (1) or copy the value to your own buffer and use it later as (2), the results are both correct. However, if you store the pointer and use it later as (3), you only get the last value change and the result is wrong.